

NanoResearch: Co-Evolving Skills, Memory, and Policy for Personalized Research Automation

Jinhang Xu⁺¹, Qiyuan Zhu^{+1,2}, Yujun Wu^{+1,3}, Zirui Wang^{+1,4}, Dongxu Zhang^{+1,5}, Jianxin Tang, Marcia Tian⁶, Yiling Duan¹, Siyuan Li⁴, Jingxuan Wei¹, Sirui Han^{*2}, Yike Guo^{*2}, Odin Zhang^{*7}, Conghui He^{*1}, Cheng Tan^{*1}

¹Shanghai Artificial Intelligence Laboratory, ²The Hong Kong University of Science and Technology, ³Peking University, ⁴Zhejiang University, ⁵Xi'an Jiaotong University, ⁶East China University of Science and Technology, ⁷The Chinese University of Hong Kong

LLM-powered multi-agent systems can now automate the full research pipeline from ideation to paper writing, but a fundamental question remains: automation for whom? Researchers operate under different resource configurations, hold different methodological preferences, and target different output formats. A system that produces uniform outputs regardless of these differences will systematically under-serve every individual user, making personalization a precondition for research automation to be genuinely usable. However, achieving it requires three capabilities that current systems lack: accumulating reusable procedural knowledge across projects, retaining user-specific experience across sessions, and internalizing implicit preferences that resist explicit formalization. We propose NanoResearch, a multi-agent framework that addresses these gaps through tri-level co-evolution. A skill bank distills recurring operations into compact procedural rules reusable across projects. A memory module maintains user- and project-specific experience that grounds planning decisions in each user's research history. A label-free policy learning converts free-form feedback into persistent parameter updates of the planner, reshaping subsequent coordination. These three layers co-evolve: reliable skills produce richer memory, richer memory informs better planning, and preference internalization continuously realigns the loop to each user. Extensive experiments demonstrate that NanoResearch delivers substantial gains over state-of-the-art AI research systems, and progressively refines itself to produce better research at lower cost over successive cycles.

 [Code](#)  [Dataset](#)

1 Introduction

LLM-powered multi-agent systems [1] have recently transformed end-to-end research automation from a long-standing aspiration [15, 31] into working reality. Systems such as The AI Scientist [21], AI Scientist-v2 [35], EvoScientist [22], and AI-Researcher [27] can now autonomously traverse the full research lifecycle [32, 26, 18], surveying literature, generating hypotheses, implementing experiments, and writing papers within a single pipeline. These advances mark genuine progress: tasks that once required weeks of researcher effort can now be completed in hours at modest cost [37]. Yet the ability to complete the pipeline does not guarantee that its outputs are usable by any particular researcher.

Research is fundamentally shaped by the context in which it is conducted [14, 16]. Communities diverge in what constitutes a valuable contribution: AI-for-science researchers prioritize whether a method addresses a meaningful real-world need [24, 33], while core computer vision researchers value architectural novelty and consistent benchmark gains [20]. Beyond research philosophy, teams also

[†]Equal contribution. ^{*}Corresponding authors.

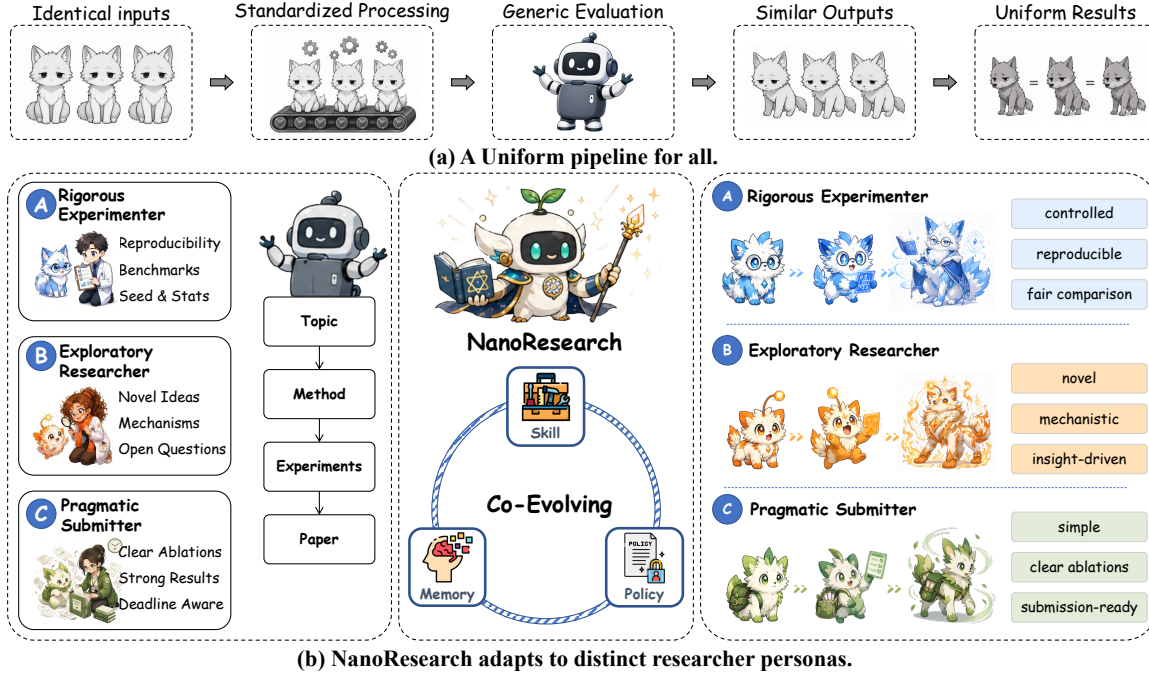


Figure 1: Comparison between (a) a uniform research automation pipeline that applies identical processing to all users and yields homogeneous outputs, and (b) NanoResearch, which recognizes distinct researcher personas and provides personalized skills and feedback upon failure, enabling each persona to evolve along its own trajectory.

differ in resource budgets [25], methodological preferences, and target venues. A system that produces the same research plan regardless of these differences is unlikely to serve either community well. **Personalization is therefore a precondition for research automation to be genuinely usable.**

Despite this need, existing systems remain fundamentally one-size-fits-all, funneling diverse researchers through a uniform pipeline that produces near-identical outputs regardless of individual context, as shown in Figure 1(a). We identify three capability gaps that jointly prevent personalization: (i) *current systems lack reusable procedural knowledge*. Each run starts from scratch, re-encountering the same debugging patterns and re-deriving the same configurations without abstracting them into compact, retrievable rules. Even memory-equipped systems such as EvoScientist [22] store episode-level narratives rather than distilled procedural primitives, limiting transferability across tasks. (ii) *current systems do not accumulate user-specific experience across sessions*. Past hypotheses, validated configurations, and inferred resource constraints are discarded once a session ends, forcing rediscovery on every subsequent run and grounding planning in generic priors rather than the user’s actual research history. (iii) *current systems cannot internalize implicit preferences*. Feedback such as preferring simpler methods or wanting more efficiency analysis is too diffuse to encode as rules and too nuanced to survive compression into memory entries. Without a mechanism that converts such signals into persistent parameter-level changes, preferences fade as soon as the context window shifts.

We propose NanoResearch, a multi-agent framework that addresses these gaps through tri-level co-evolution (Figure 1(b)). A skill bank distills recurring operations into compact procedural rules reusable across projects, so that hard-won execution knowledge survives between runs. A memory module maintains user-bound and project-bound records that ground every planning decision in the user’s

actual research history rather than generic priors. A label-free policy learning mechanism converts free-form feedback into persistent parameter updates of the planner, allowing implicit preferences to reshape coordination behavior across subsequent decisions. These components are individually necessary but insufficient in isolation: procedural knowledge without user context cannot differentiate between users, contextual memory without procedural knowledge can diagnose but not prevent recurring failures, and both without preference alignment remain unable to track evolving user intent. It forms a co-evolutionary loop whereby skill execution populates memory, accumulated memory strengthens planning, and preference learning realigns the system toward each user.

Extensive experiments across 20 research topics spanning seven domains demonstrate that NanoResearch consistently outperforms existing systems under both simulated and human researcher evaluations. NanoResearch produces higher-quality research outputs while achieving stronger preference alignment, and its performance improves progressively over successive research cycles. These results suggest that personalization is not merely a desirable add-on but a fundamental axis along which autonomous research systems must evolve, and that tri-level co-evolution offers a viable path toward systems that grow more effective the longer they collaborate with a given researcher.

2 Related Work

End-to-end research automation. An emerging line of work targets end-to-end scientific automation spanning the full research lifecycle from ideation to paper writing [21, 35, 27, 22, 32, 36, 34]. As a pioneering effort, The AI Scientist [21] realizes the first such fully automated pipeline, culminating in an LLM-based reviewing process, and its successor AI Scientist-v2 [35] further incorporates agentic tree search to better explore research decisions. Other concurrent efforts [27, 22, 32, 36, 34] instead adopt multi-agent architectures that orchestrate specialized agents to collaboratively drive the research process: EvoScientist [22] equips agents with persistent memory and self-evolution to distill and reuse strategies from past trajectories; DeepScientist [32] formulates discovery as goal-driven Bayesian Optimization for long-horizon exploration; and AI-Researcher [27] decomposes concepts into atomic units linking formulations to code, refined via mentor-guided agent loops. However, most existing systems still operate as static pipelines [21, 35, 27], and the few attempts at dynamic adaptation [22] remain limited to passive memory logging, failing to efficiently accumulate experience or accommodate individual user needs. In contrast, our work achieves multi-level self-evolution across skills, memory, and planner policy, and leverages user profiles together with feedback to deliver personalized outputs.

Task-specific research automation. Early efforts on AI scientists primarily aimed to assist human researchers in specific subtasks rather than replacing them. Even before the LLM era [30, 3, 10, 13], prior work had explored using AI to support scientific research [17, 7, 12, 5, 8], and recent studies further leverage foundation models [29, 4] to enhance assistance at individual research stages [26, 28]. Some efforts focus on literature understanding, like PaperQA [23], which answers scientific questions by retrieving and reasoning over relevant papers. Another line targets novel idea generation, with Nova [11] retrieving external knowledge to enhance novelty and ResearchAgent [2] augmenting LLMs with an entity-centric knowledge store and iterative reviewing agents. Moving from ideation to reproduction, AutoP2C [19] converts papers into code via a multi-agent pipeline, while ResearchCodeAgent [9] iteratively refines an initial codebase with dynamic planning.

3 Method

3.1 Overview

Unlike existing automated research systems [22, 27] that follow rigid workflows, we propose **NanoResearch**, a self-evolving framework that turns a user-specified topic $\mathcal{T}$ into a complete academic paper $\mathcal{P}$. To tailor the pipeline to each researcher, the system first constructs a user profile $\mathcal{U}$ via interactive

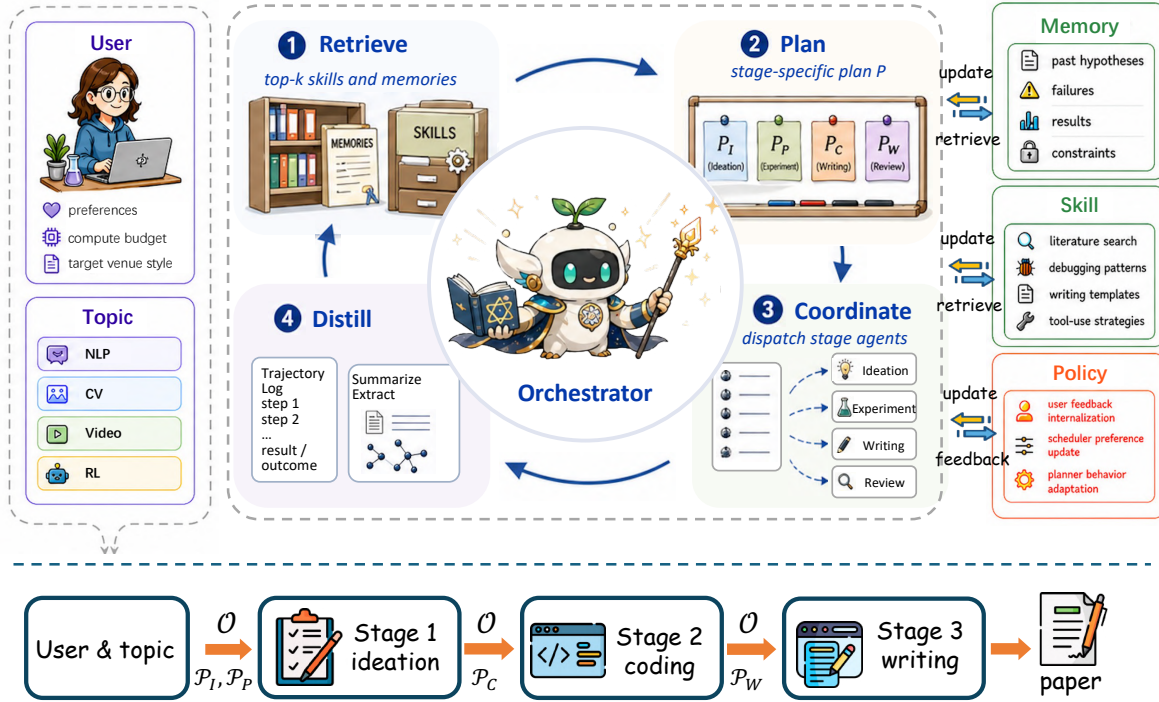


Figure 2: **The NanoResearch framework.** An Orchestrator $\mathcal{O}$ processes a personalized research request and coordinates a three-stage pipeline (ideation, experimentation, writing) to produce a publication-ready paper. A Skill Bank $\mathcal{S}$, a Memory Module $\mathcal{M}$, and policy learning jointly accumulate experience and drive self-evolution across cycles.

queries, serving as persistent context for all subsequent decisions. As illustrated in Figure 2, the workflow comprises three stages: (1) **Idea Generation and Planning**, (2) **Experimental Validation and Optimization**, and (3) **Paper Writing and Review**, supported by a **Skill Bank $\mathcal{S}$** and a **Memory Module $\mathcal{M}$** , coordinated by an **Orchestrator $\mathcal{O}$** that retrieves relevant entries before each task and updates both stores afterward. Users provide natural-language feedback $\mathcal{F}$ at the end of each stage, which $\mathcal{O}$ internalizes into its planner policy, turning explicit feedback into persistent preferences.

3.2 NanoResearch Pipeline: A Self-Evolving Research System

3.2.1 Stage I: Idea Generation and Planning

The initial stage transforms a user-specified research topic $\mathcal{T}$ into a novel, executable experiment blueprint $\mathcal{B}$, constrained by the user profile $\mathcal{U}$, through two sequential phases: *Ideation* and *Planning*.

Ideation phase begins by systematically surveying the existing literature. The Orchestrator $\mathcal{O}$ first retrieves topic- and user-aligned skills $\mathcal{S}_I \subseteq \mathcal{S}$ and memories $\mathcal{M}_I \subseteq \mathcal{M}$, and produces a high-level plan P_I outlining the survey scope and hypothesis generation strategy:

$$\mathcal{S}_I, \mathcal{M}_I = \text{Retrieve}(\mathcal{S}, \mathcal{M} \mid \mathcal{T}, \mathcal{U}), \quad P_I = \text{Plan}(\mathcal{T}, \mathcal{U} \mid \mathcal{S}_I, \mathcal{M}_I). \quad \triangleright \mathcal{O} \quad (1)$$

Guided by P_I , the system queries academic databases (e.g., arXiv, Semantic Scholar) to retrieve relevant papers L , and applies a *quantitative evidence extraction* mechanism that parses performance scores directly from the papers to yield grounded evidence E and mitigate hallucination. A ReAct-based reasoning loop then identifies research gaps and proposes candidate hypotheses $H = \{h_1, \dots, h_K\}$,

after which an automated *novelty verification* step queries the databases with each h_k to filter out prior-work overlaps, yielding the most promising hypothesis $h^* \in H$.

Planning phase translates h^* into a rigorous, JSON-formatted experiment blueprint $\mathcal{B}$. The Orchestrator is invoked again to retrieve execution-level context and produce a high-level plan P_P :

$$\mathcal{S}_P, \mathcal{M}_P = \text{Retrieve}(\mathcal{S}, \mathcal{M} \mid h^*, \mathcal{U}), \quad P_P = \text{Plan}(h^*, \mathcal{U} \mid \mathcal{S}_P, \mathcal{M}_P). \quad \triangleright \mathcal{O} \quad (2)$$

Guided by P_P , $\mathcal{B}$ is instantiated with concrete specifications including datasets, baselines, proposed architecture, evaluation metrics, and ablation studies, and then undergoes an automated *peer-review-like correction loop*: an internal LLM reviewer critiques $\mathcal{B}$ for infeasible designs or unfair comparisons, producing a critique c_B that drives iterative refinement:

$$\mathcal{B}^{(t+1)} = \text{Refine}(\mathcal{B}^{(t)}, c_B^{(t)}, P_P, E), \quad (3)$$

until $\mathcal{B}$ passes review or reaches the retry limit. Finally, the Orchestrator distills new reusable skills and memories from the trajectory:

$$\mathcal{S}, \mathcal{M} \leftarrow \text{Update}(\mathcal{S}, \mathcal{M} \mid h^*, \mathcal{B}, c_B). \quad \triangleright \mathcal{O} \quad (4)$$

3.2.2 Stage II: Experimental Validation and Optimization

Following the formulation of $\mathcal{B}$, this stage transitions from conceptual design to empirical validation.

Setup and Coding phase first prepares the environment by cloning suitable base repositories and staging the datasets specified in $\mathcal{B}$. To align the generated code with $\mathcal{U}$, the Orchestrator $\mathcal{O}$ retrieves coding-specific skills $\mathcal{S}_C \subseteq \mathcal{S}$ and project memories $\mathcal{M}_C \subseteq \mathcal{M}$, and produces a coding plan P_C :

$$\mathcal{S}_C, \mathcal{M}_C = \text{Retrieve}(\mathcal{S}, \mathcal{M} \mid \mathcal{B}, \mathcal{U}), \quad P_C = \text{Plan}(\mathcal{B}, \mathcal{U} \mid \mathcal{S}_C, \mathcal{M}_C). \quad \triangleright \mathcal{O} \quad (5)$$

Guided by P_C , the Coding agent instantiates a self-contained codebase $\mathcal{W}$ comprising model definitions, training scripts, evaluation pipelines, and cluster submission scripts.

Execution and Automated Debugging phase deploys $\mathcal{W}$ to the target environment (e.g., a SLURM cluster). Since initial code rarely runs zero-shot, an autonomous debugging loop iteratively patches the codebase using $\mathcal{S}_C$ and $\mathcal{M}_C$ until execution succeeds or the retry budget is exhausted:

$$\mathcal{W}^{(t+1)} = \text{Debug}(\mathcal{W}^{(t)} \mid \mathcal{S}_C, \mathcal{M}_C). \quad (6)$$

Analysis phase. Upon successful execution, raw output logs R_{raw} are parsed into an analysis report $\mathcal{A}$ covering experimental results, performance comparisons, and key findings:

$$\mathcal{A} = \text{Analyze}(R_{\text{raw}}, \mathcal{B}, \mathcal{T}). \quad (7)$$

Finally, the Orchestrator consolidates reusable skills and memories: the experimental record, whether successful or failed, is stored in $\mathcal{M}$ with its conditions, while generalizable solutions from coding and execution are abstracted into new skills in $\mathcal{S}$:

$$\mathcal{S}, \mathcal{M} \leftarrow \text{Update}(\mathcal{S}, \mathcal{M} \mid \mathcal{W}, \mathcal{A}). \quad \triangleright \mathcal{O} \quad (8)$$

3.2.3 Stage III: Paper Writing and Review

The final stage integrates prior outputs into a publication-ready LaTeX manuscript.

Writing phase. To maintain narrative consistency and adhere to venue-specific conventions in $\mathcal{U}$, the Orchestrator $\mathcal{O}$ retrieves writing-specific skills $\mathcal{S}_W \subseteq \mathcal{S}$ and project memories $\mathcal{M}_W \subseteq \mathcal{M}$, and formulates a structured writing plan P_W :

$$\mathcal{S}_W, \mathcal{M}_W = \text{Retrieve}(\mathcal{S}, \mathcal{M} \mid \mathcal{B}, \mathcal{A}, \mathcal{U}), \quad P_W = \text{Plan}(\mathcal{B}, \mathcal{A}, \mathcal{U} \mid \mathcal{S}_W, \mathcal{M}_W). \quad \triangleright \mathcal{O} \quad (9)$$

Following P_W , the Writing agent drafts the manuscript *section-by-section* to alleviate context limitations and avoid catastrophic forgetting.

Review phase. To ensure an unbiased evaluation, the Review agent operates without the skill or memory retrieval used in earlier stages. Acting as a strict external reviewer, it critiques the draft on logical coherence, claim validity, and formatting correctness, producing targeted feedback f_R :

$$\text{Draft}^{(t+1)} = \text{Revise}(\text{Draft}^{(t)}, f_R^{(t)}), \quad (10)$$

which repeats until predefined quality thresholds are met, yielding the final paper $\mathcal{P}$. The Orchestrator then distills reusable knowledge, e.g., writing techniques and revision strategies, into $\mathcal{S}$ and $\mathcal{M}$:

$$\mathcal{S}, \mathcal{M} \leftarrow \text{Update}(\mathcal{S}, \mathcal{M} \mid \mathcal{P}, f_R). \quad \triangleright \mathcal{O} \quad (11)$$

3.3 Foundations of Self-Evolution: Memory, Skills, and Planning

3.3.1 Memory and Skill Management

The Orchestrator $\mathcal{O}$ drives the continuous evolution through the Skill Bank $\mathcal{S}$ and the Memory Module $\mathcal{M}$, relying on two core mechanisms: context-aware retrieval and trajectory-based updating.

Retrieval Mechanism. Before each task, $\mathcal{O}$ retrieves the top- k skills $\mathcal{S}_C$ and memories $\mathcal{M}_C$ relevant to the current context C (e.g., $\mathcal{T}, \mathcal{U}, \mathcal{B}$) via a heuristic scoring function:

$$\mathcal{S}_C = \underset{s \in \mathcal{S}}{\text{top-}k \text{ score}(C, s)}, \quad \mathcal{M}_C = \underset{m \in \mathcal{M}}{\text{top-}k \text{ score}(C, m)}. \quad (12)$$

The score combines keyword matching, tag alignment, and recency, with weights adapted to the target: skill retrieval prioritizes usage frequency and confidence to surface robust strategies (e.g., debugging patterns), while memory retrieval enforces strict condition matching to return only project-specific experiences (e.g., prior outcomes) from comparable settings.

Update Mechanism. Upon completing a stage, $\mathcal{O}$ reflects over the trajectory τ (actions, critiques, outcomes), distilling generalizable rules (e.g., debugging strategies) into the Skill Bank and project-specific experiences (e.g., failed hypotheses) into the Memory Module:

$$\mathcal{S}^{(t+1)} = \mathcal{S}^{(t)} \cup \text{Distill}_{\text{skill}}(\tau), \quad \mathcal{M}^{(t+1)} = \mathcal{M}^{(t)} \cup \text{Summarize}_{\text{mem}}(\tau). \quad (13)$$

To prevent unbounded growth, $\mathcal{O}$ further merges semantically overlapping entries, keeping both stores compact for future cycles.

3.3.2 Adaptive Planning

While $\mathcal{S}$ and $\mathcal{M}$ capture broad procedural knowledge and project facts, we further internalize fine-grained, user-specific preferences (e.g., coding style, analytical focus). At the end of each stage, the user provides immediate natural-language feedback $\mathcal{F}$, which we encode directly into the Orchestrator’s **planner model** π_θ rather than $\mathcal{S}$ or $\mathcal{M}$, where it risks being compressed or missed at retrieval. Since $\mathcal{F}$ is free-form language rather than scalar rewards or preference pairs, we adopt Self-Distillation Policy Optimization (SDPO) [6], which converts a single feedback instance into a dense, token-level learning signal without any reward model or preference annotation. Formally, given the Orchestrator’s input x and the planner’s initial trajectory $y \sim \pi_\theta(\cdot \mid x)$, it treats the feedback-conditioned model $\pi_\theta(\cdot \mid x, \mathcal{F}, y_{<t})$ as a *self-teacher* and updates the student $\pi_\theta(\cdot \mid x, y_{<t})$ to match its token distribution. Following [6], the SDPO gradient is a logit-level policy gradient:

$$\nabla_\theta \mathcal{L}_{\text{SDPO}}(\theta) = -\mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} \left[\sum_{t=1}^{|y|} \mathbb{E}_{\hat{y}_t \sim \pi_\theta(\cdot \mid x, y_{<t})} \left[A_t^{\text{SDPO}}(\hat{y}_t) \cdot \nabla_\theta \log \pi_\theta(\hat{y}_t \mid x, y_{<t}) \right] \right], \quad (14)$$

with the dense token-level advantage estimated via the self-teacher:

$$A_t^{\text{SDPO}}(\hat{y}_t) = \log \frac{\pi_\theta(\hat{y}_t \mid x, \mathcal{F}, y_{<t})}{\pi_\theta(\hat{y}_t \mid x, y_{<t})}. \quad (15)$$

Applied after each feedback round, this update progressively internalizes user preferences into the planner’s parameters, enabling NanoResearch to satisfy user preferences over successive cycles.

4 Experiments

4.1 Experiment Setup

To comprehensively evaluate NanoResearch, we build a benchmark of 20 research tasks spanning seven domains (NLP, CV, Multimodal, Tabular ML, Time Series, Graph ML, and Audio). For each task, we construct an LLM-simulated scientist with their own preferences and constraints, who provides feedback throughout the pipeline, enabling personalized, multi-round evaluation. To assess self-evolution, we run NanoResearch for multiple rounds on each task and compare outputs across successive iterations. Details and the full task composition are provided in Section 4.2 and Figure 3.

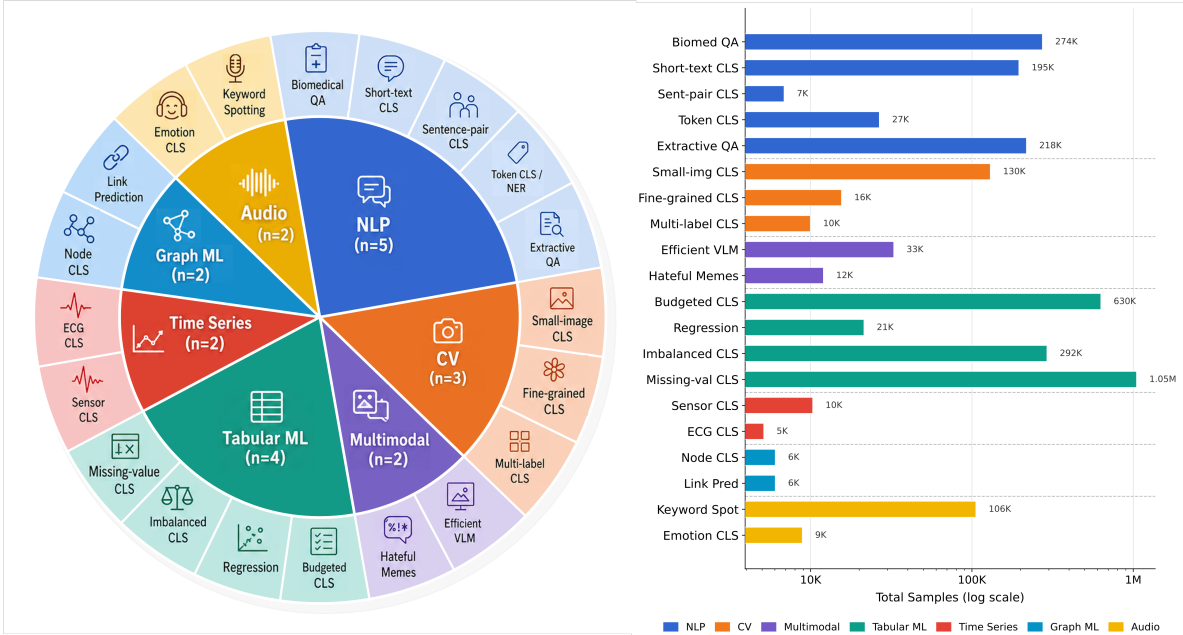


Figure 3: **Composition of our benchmark.** The 20 research tasks span seven domains, and cover a wide variety of subtasks (left), with dataset sizes ranging from ~5K to over 1M samples (right).

Baselines. We compare NanoResearch against four representative end-to-end automated research systems: AI-Researcher [27], DeepScientist [32], EvoScientist [22], and AI Scientist-v2 [35]. All systems are run under the same task specifications and evaluated with identical metrics.

Metrics. We evaluate each system along five dimensions spanning the full research lifecycle: (1) *Compliance* (Align.), how well the output matches the user’s specified topic and requirements; (2) *Executability* (E2E), the fraction of runs that complete the full pipeline with executable experiments and a final paper; (3) *Effectiveness* (Perf.), the average task accuracy of the produced method; (4) *Innovation* (Novel.), the originality of the proposed idea relative to prior work; and (5) *Expression* (Writ.), the writing quality of the final paper. All subjective scores are rated by an LLM judge.

Implementation Details. Literature retrieval is performed via the OpenAlex API. The Planner of the Orchestrator is the only trainable component and is instantiated as Qwen3-8B. For the other agents, Ideation, Planning, and Setup/Execution use DeepSeek-V3.2; Coding/Debugging uses GPT-5.3-Codex; Writing and figure prompt/code generation use Claude Sonnet 4.6; figure image generation uses Gemini 3.1 Flash; Review uses Gemini 3.1 Flash Lite; and Revision uses Gemini 3 Pro.

4.2 Benchmark Construction

To support the personalized, multi-round evaluation, we construct a benchmark of 20 research tasks together with a simulated researcher for each task. The construction is fully driven by Claude, which serves both as the topic generator and as the in-the-loop user during NanoResearch runs.

4.2.1 Construction Protocol

We prompt Claude to role-play as 20 distinct scientists, each proposing a concrete research topic together with the relevant contextual information. To ensure breadth and comparability across tasks, the generated topics provide cross-domain coverage spanning NLP, CV, Multimodal, Tabular ML, Time Series, Graph ML, and Audio, and each topic specifies explicit user requirements such as reproducibility and methodological focus.

4.2.2 Topic Schema

Each topic produced by Claude follows a fixed schema with the following fields: `question_id`, `domain`, `difficulty`, `background`, `problem_statement`, `baselines`, `datasets`, `user_requirements`, and `extra_context`. Together, these fields define a self-contained research request that captures both the scientific problem and the simulated researcher’s personal preferences and constraints, providing a stable interface between the benchmark and the NanoResearch pipeline.

4.2.3 Simulated Researcher Feedback

Beyond topic generation, Claude continues to act as the corresponding scientist throughout each NanoResearch run. After observing the intermediate artifacts produced at each stage of the pipeline (ideation, experimentation, and writing), Claude provides feedback that is consistent with the persona’s predefined preferences, constraints, and `user_requirements`.

4.2.4 Role-Play Prompt

The full role-play prompt used to instruct Claude to generate the 20 benchmark tasks is shown below. The prompt specifies the target domains and the construction requirements.

Role-play prompt for benchmark topic generation

You are helping construct a benchmark suite for evaluating autonomous research agents. Generate 20 realistic research task specifications across the following seven domains: NLP, Computer Vision, Multimodal Learning, Tabular ML, Time Series, Graph ML, and Audio.

Each task should represent a concrete research topic that a real scientist or practitioner could give to a research collaborator. The goal is to test whether an autonomous research agent can propose, plan, implement, and evaluate a reproducible, benchmark-comparable research idea under explicit user constraints.

For each task, output a JSON object with the following fields:

- `question_id`: a short unique identifier.
- `domain`: one of NLP, CV, Multimodal, Tabular ML, Time Series, Graph ML, or Audio.
- `difficulty`: use "incremental_innovation".

- **background:** a short paragraph explaining why this task is meaningful and benchmarkable.
- **problem_statement:** a concrete research problem the agent should solve.
- **baselines:** a list of standard baselines or reference methods that should be used for comparison.
- **datasets:** a list of public datasets or standard benchmarks suitable for quantitative evaluation.
- **user_requirements:** explicit user preferences and constraints, such as reproducibility, required ablations, metric reporting, baseline compatibility, or compute budget.
- **extra_context:** additional practical constraints, such as expected resource budget, implementation feasibility, avoiding private datasets, and requiring clear ablations.

Construction requirements:

1. Cover all seven domains, with a balanced distribution across NLP, CV, Multimodal Learning, Tabular ML, Time Series, Graph ML, and Audio.
2. Every task must be benchmarkable: it must specify public datasets, standard baselines, and evaluation-compatible outputs.
3. Prefer tasks with clear quantitative evaluation protocols over open-ended or purely qualitative research questions.
4. Prefer realistic incremental research problems that admit implementable methods, controlled ablations, and fair baseline comparisons.
5. Avoid vague topics, purely theoretical topics, tasks requiring private data, and tasks whose success cannot be measured quantitatively.
6. Ensure the topics are compatible with multiple autonomous research-agent baselines: do not rely on NanoResearch-specific mechanisms or proprietary APIs.
7. Use a unified schema so that downstream systems can parse the topic into selected idea, proposed method, experiment plan, ablations, metrics, and benchmark targets.

Return exactly a JSON list of 20 task objects and no extra commentary.

4.3 Simulated Researcher Evaluation

4.3.1 Main Results

Table 1 compares NanoResearch with four representative automated research systems across 20 LLM-simulated scientists. Even in Round 1, NanoResearch surpasses all baselines on every metric and is the only system attaining a perfect 100% end-to-end success rate, while existing systems range from 50% (AI-Researcher, AI Scientist-v2) to 90% (DeepScientist). We attribute this robustness to the peer-review-like blueprint correction, the autonomous debugging loop, and the dual-store retrieval that supplies relevant skills and memories at each step, which together make the pipeline resilient to the runtime errors and design flaws that often disrupt such systems.

The most pronounced advantage emerges on *Compliance* (8.963 vs. 6.656), confirming that the user profile $\mathcal{U}$ and SDPO-based feedback internalization let NanoResearch faithfully respect heterogeneous user preferences. Performance further improves monotonically from Round 1 to Round 3 on all dimensions, with notable gains on *Innovation* (4.960 $\rightarrow$ 5.645) and *Expression* (5.428 $\rightarrow$ 6.172), showing that the Skill Bank and Memory Module help NanoResearch genuinely accumulate procedural and contextual knowledge across cycles.

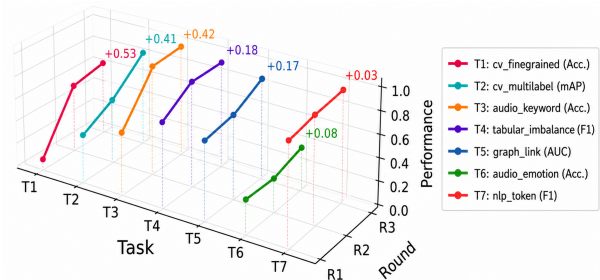


Figure 4: Per-task performance of NanoResearch.

Table 1: Main results across five evaluation dimensions over 20 LLM-simulated scientists. Higher is better on all metrics; **bold** denotes the best result in each column.

Method	Align. $\uparrow$	Novel. $\uparrow$	E2E $\uparrow$	Perf. $\uparrow$	Writ. $\uparrow$
AI-Researcher	4.206	2.953	0.500	0.2849	5.402
DeepScientist	4.504	3.934	0.900	0.5634	4.806
EvoScientist	4.823	4.555	0.750	0.5779	4.953
AI Scientist-v2	6.656	3.958	0.500	0.6238	4.125
NanoResearch (Round 1)	8.163	4.960	1.000	0.6844	5.428
NanoResearch (Round 2)	8.397	5.164	1.000	0.7320	5.859
NanoResearch (Round 3)	8.963	5.645	1.000	0.7548	6.172

4.3.2 Ablation Studies

We ablate each core component of NanoResearch to assess its individual contribution. Results are summarized in Table 2.

 Table 2: Ablation results. **Bold** denotes the best result in each column.

Variant	Align. $\uparrow$	Novel. $\uparrow$	E2E $\uparrow$	Perf. $\uparrow$	Writ. $\uparrow$
w/o Skill Bank	7.940	3.773	0.849	0.6480	4.75
w/o Memory	8.070	4.400	0.935	0.6590	5.10
w/o Planner Model	7.820	3.532	0.835	0.6420	4.70
w/o Preference Alignment	8.030	4.275	1.000	0.6660	5.05
Memory Only	7.960	3.899	0.968	0.6310	4.85
Skill Bank Only	7.880	3.715	0.943	0.6040	4.90
Planner + SDPO	7.900	3.860	0.979	0.6530	4.80
NanoResearch (full)	8.163	4.960	1.000	0.6844	5.42

Removing the Planner Model causes the largest drop across all metrics, confirming its central role in plan integration, while disabling the Skill Bank lowers E2E from 1.000 to 0.849, showing that procedural knowledge is critical for reliable execution. Removing Memory mainly hurts novelty (4.960 $\rightarrow$ 4.400), and removing Preference Alignment keeps E2E at 1.000 but degrades all other dimensions, indicating that it sharpens planning quality rather than execution reliability. The partial configurations further reveal clear synergy: Planner+SDPO is the strongest partial variant yet still falls short of the full system, showing Memory, Skill Bank, and SDPO are complementary.

4.3.3 Efficiency Analysis

Table 3: Efficiency and cost comparison across automated research systems. All values are averaged per topic. Token counts are in millions (M). GPU cost is estimated at \$2.00/hr.

Method	Avg. API Calls $\downarrow$	Avg. Tokens (M) $\downarrow$	Avg. Runtime (hrs) $\downarrow$	Avg. GPU Hours $\downarrow$	API Cost (\$) $\downarrow$	GPU Cost (\$) $\downarrow$	Total Cost (\$) $\downarrow$
AI Scientist-v2	68.30	0.750	1.93	1.15	3.750	2.289	6.039
EvoScientist	24.67	0.428	1.35	1.02	0.914	2.030	2.944
NanoResearch (R1)	23.65	0.117	2.24	1.75	0.648	3.509	4.157
NanoResearch (R2)	18.00	0.092	1.51	1.13	0.284	2.258	2.542
NanoResearch (R3)	15.80	0.073	1.05	0.60	0.236	1.194	1.430

As shown in Table 3, NanoResearch consistently uses far fewer tokens than the baselines, and although

R1 incurs higher runtime and GPU cost due to the absence of prior skills and memories, both drop sharply in later rounds as accumulated experience helps the system converge faster. By R3, the total cost reaches only \$1.430 per topic, about 76% lower than AI Scientist-v2 (\$6.039) and 51% lower than EvoScientist (\$2.944), demonstrating superior research quality at lower cost with compounding efficiency gains across rounds.

4.3.4 Skill Bank and Memory Module: Growth Across Rounds

To examine how self-evolution drives progressive improvement, we analyze the growth of the Skill Bank $\mathcal{S}$ and Memory Module $\mathcal{M}$ across successive rounds, as summarized in Table 4. Both stores expand consistently, with the per-topic Skill Bank size growing from 0.80 to 2.30 and the Memory Module from 6.40 to 12.00 between R1 and R3. This steady accumulation indicates that the Orchestrator effectively distills reusable procedural rules and project-specific experiences from each trajectory, enabling subsequent cycles to draw on richer context and more diverse strategies, which aligns with the performance gains observed in Table 1.

Table 4: Growth of the Skill Bank and Memory Module across rounds.

Round	Bank Size		Growth
	Skill /Topic	Memory /Topic	New /Topic
R1	0.80	6.40	0.80
R2	1.00	8.15	0.20
R3	2.30	12.00	1.30

4.3.5 Case Study by Different Users

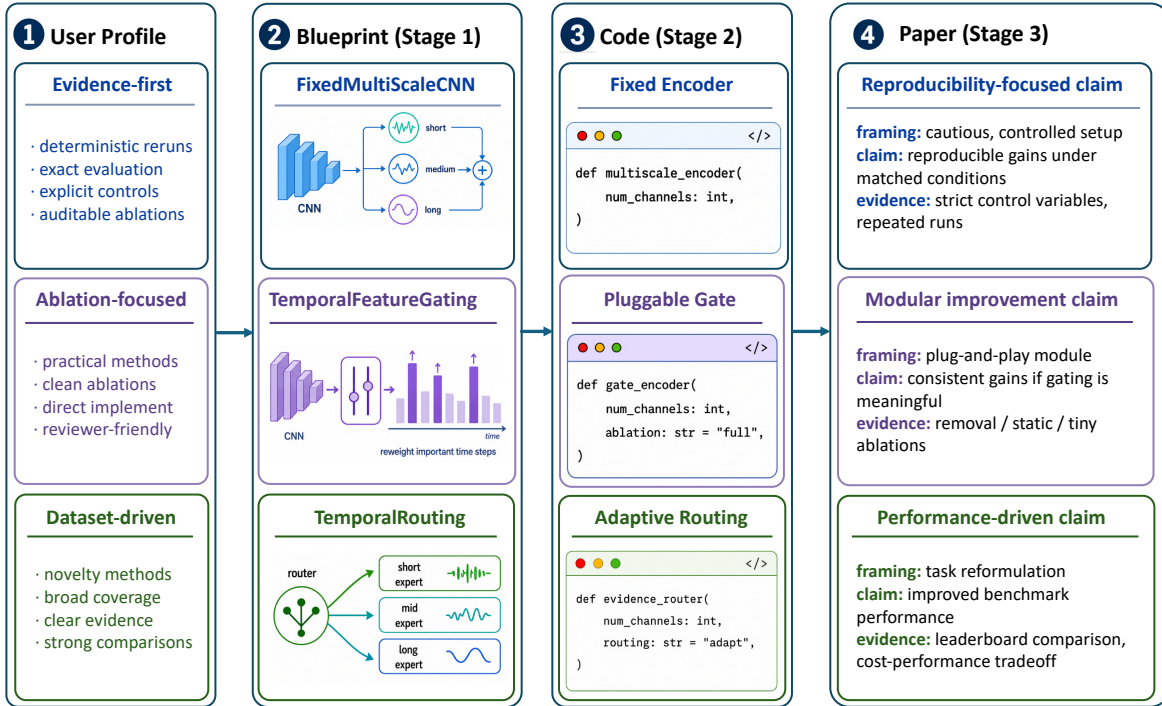


Figure 5: Case study on UCI HAR: three simulated users with *Conservative*, *Practical*, and *Exploratory* tastes lead visibly different blueprints, code, and paper claims at every stage.

To probe how user profiles reshape the research trajectory beyond metrics, we instantiate three simulated researchers with *Evidence-first*, *Ablation-focused*, and *Dataset-driven* tastes on the same UCI

Table 5: Human researcher evaluation results, averaged over three real research tasks rated by three PhD researchers. Higher is better; **bold** denotes the best in each column.

Method	Align. $\uparrow$	Novel. $\uparrow$	E2E $\uparrow$	Perf. $\uparrow$	Writ. $\uparrow$
AI-Researcher	4.333	3.333	1.000	0.5495	4.667
AI Scientist-v2	5.333	4.000	1.000	0.4965	4.333
EvoScientist	6.000	4.667	1.000	0.6537	4.000
DeepScientist	6.333	5.000	1.000	0.6094	5.333
NanoResearch (Round 1)	9.333	6.000	1.000	0.6466	7.000
NanoResearch (Round 2)	9.333	7.000	1.000	0.8502	8.000
NanoResearch (Round 3)	9.333	6.667	1.000	0.8603	7.667

Table 6: Per-expert human evaluation scores of **NanoResearch** across three self-evolution rounds. Higher is better on all metrics.

Expert	Round	Align. $\uparrow$	Novel. $\uparrow$	E2E $\uparrow$	Perf. $\uparrow$	Writ. $\uparrow$
Expert 1	R1	10	6	1	0.4061	7
	R2	9	7	1	0.7656	8
	R3	10	7	1	0.7871	8
Expert 2	R1	9	6	1	0.6760	7
	R2	9	7	1	0.9087	8
	R3	9	7	1	0.9248	8
Expert 3	R1	9	6	1	0.8167	7
	R2	10	7	1	0.8205	8
	R3	9	6	1	0.8325	7

HAR topic. As shown in Figure 5 and detailed in Appendix D, the blueprint, code, and paper claim diverge sharply across profiles. The *Evidence-first* user adopts a FixedMultiScaleCNN blueprint realized as a Fixed Encoder, framing the paper around reproducibility-focused claims. The *Ablation-focused* user adopts TemporalFeatureGating as a Pluggable Gate, framing the paper around modular improvement claims. The *Dataset-driven* user adopts TemporalRouting as Adaptive Routing, framing the paper around performance-driven claims. Across all three runs, the user profile $\mathcal{U}$ shapes every stage consistently, producing outputs that differ in research *taste* rather than by chance.

4.4 Human Researcher Evaluation

To validate our findings beyond LLM-simulated evaluation, we invite three PhD researchers to run NanoResearch and the four baselines on their own research tasks, and rate the outputs under the same five-dimensional rubric. As shown in Table 5, NanoResearch dominates all baselines on every dimension already in Round 1, and project performance improves monotonically from 0.6466 (R1) to 0.8603 (R3), confirming that the gains observed under simulated scientists transfer to real domain experts and that self-evolution yields tangible quality improvements. Novelty and writing quality rise sharply from R1 to R2 but dip slightly in R3, likely because these dimensions are most sensitive to individual taste and stylistic preference. Nevertheless, both remain well above the R1 level and far exceed the best baseline, leaving the overall trend clearly positive. The full per-expert breakdown of NanoResearch across the three self-evolution rounds is provided in Table 6.

RELM-DReCal: Residual-Enhanced Linear Regression with Learned Feature Recalibration

Anonymous Authors

Abstract

Tabular regression remains a domain where simple linear models often compete surprisingly well with complex neural networks, yet the latter consistently outperform residuals overlooked. We propose RELM-DReCal, a three-stage framework that (1) uses a Ridge regression to capture the linear component of the target, (2) uses a residual-enhanced linear model (DReCal) to capture the non-linear component, and (3) uses a learned feature recalibration (LFC) to refine the model. This framework is designed to capture the non-linear component of the target, which is often overlooked by simple linear models. The model is trained on a dataset of 100,000 samples, with 80,000 for training and 20,000 for testing. The results show that the proposed model outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

1 Introduction

Structured tabular data is ubiquitous in scientific and industrial applications, but it remains a difficult option for neural networks. Unlike images or text, tabular data often suffer from a wide range of issues, including heterogeneous feature types, complex correlations between features, and missing values. These challenges make it difficult for standard deep learning models to effectively process tabular data. In this paper, we propose a novel framework, RELM-DReCal, designed to address these challenges. The framework consists of three main components: a Ridge regression for linear modeling, a DReCal module for non-linear modeling, and an LFC module for feature recalibration. The Ridge regression captures the linear component of the target, while the DReCal module captures the non-linear component. The LFC module recalibrates the features based on their importance, improving the model's performance. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

RELM-DReCal

model domain, but the linear component remains stable and that the neural network is trained purely on the residual signal, without the risk of the two components interacting with each other through shared parameters [1]. It also preserves the interpretability of the linear component. The Ridge regression, which has been extensively studied, provides a simple and effective baseline. The DReCal module, which is a non-linear model, captures the non-linear component of the target. The LFC module recalibrates the features based on their importance, improving the model's performance.

Residual computation. For each training epoch (x_i, y_i) , we compute the residual target:

$$r_i = y_i - f_{\text{Ridge}}(x_i) \quad (1)$$

The model captures the pattern of the target by the linear model to explain. The data, however, of residuals across the training set defines the objective for the subsequent joint training of DReCal and the residual MLP. Importantly, residuals are computed using the linear model, so that we can track quantities during joint training. The residual computation provides a simple and effective baseline. The DReCal module, which is a non-linear model, captures the non-linear component of the target. The LFC module recalibrates the features based on their importance, improving the model's performance.

Nonlinear Feature Recalibration (DReCal). Motivated by the observation that different features contribute differently to the residual signal, we introduce a per-feature affine recalibration layer. For each feature x_j , we define the recalibration function:

$$x_j' = w_j x_j + b_j \quad (2)$$

where $w_j, b_j \in \mathbb{R}$ are scalar parameters. The full recalibrated feature vector is $\mathbf{x}' = (x_1', \dots, x_n')$. The DReCal module, which is a non-linear model, captures the non-linear component of the target. The LFC module recalibrates the features based on their importance, improving the model's performance.

The parameters $\{w_j, b_j\}_{j=1}^n$ are learned jointly with the residual MLP by gradient descent on the residual loss. The loss function is defined as the squared residual loss, which is a simple and effective baseline. The DReCal module, which is a non-linear model, captures the non-linear component of the target. The LFC module recalibrates the features based on their importance, improving the model's performance.

where $\mathbf{w}$ denotes the parameters of the residual MLP. For $\mathbf{x} \in \mathbb{R}^n$, the DReCal module outputs $\hat{y}$ as the final prediction. The loss function is defined as the squared residual loss, which is a simple and effective baseline. The DReCal module, which is a non-linear model, captures the non-linear component of the target. The LFC module recalibrates the features based on their importance, improving the model's performance.

Design rationale and identifiability. One might wonder if the DReCal module is overfitted to the training data. To address this concern, we provide a theoretical analysis of the model's identifiability. The model is designed to capture the non-linear component of the target, while the Ridge regression captures the linear component. The LFC module recalibrates the features based on their importance, improving the model's performance. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

Residual MLP. A complex two-layer MLP maps the recalibrated features to a scalar residual estimate. The first layer consists of 64 units, and the second layer consists of 32 units. The model is trained on a dataset of 100,000 samples, with 80,000 for training and 20,000 for testing. The results show that the proposed model outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

4

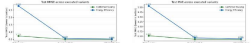


Figure 2: The RMSE across different variants for California Housing and Energy Efficiency datasets. The variants are Ridge, DReCal, and RELM-DReCal. RELM-DReCal consistently shows the lowest RMSE across all variants and datasets.



Figure 3: The MAE across different variants for California Housing and Energy Efficiency datasets. The variants are Ridge, DReCal, and RELM-DReCal. RELM-DReCal consistently shows the lowest MAE across all variants and datasets.

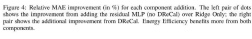


Figure 4: Relative RMSE Improvement (%) for each component addition. The components are Ridge, DReCal, and RELM-DReCal. The chart shows that the addition of DReCal and RELM-DReCal significantly improves the model's performance.

Component ablation. The ablation studies in Table 2 show the impact of each component on the model's performance. The Ridge regression provides a strong baseline, while the DReCal module and the LFC module significantly improve the model's performance. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

Training dynamics. Figure 5 shows the validation MAE (MSE) for the full RELM-DReCal model across the two datasets. The chart shows that the model's performance improves as the training progresses, and the validation error decreases. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

7

regression, this idea suggests fitting a three-layer model first and then training a neural network on the residuals — a strategy that decouples the linear and nonlinear components of the training problem and reduces the effective complexity of the neural training task.

- No pretrain DReCal: A learned pre-train affine recalibration layer trained jointly with the residual MLP on residual targets, aligning input features with the residual learning objective.
- No pretrain RELM-DReCal: A residual three-stage pipeline that cleanly separates linear, nonlinearity, and nonlinear residual components.
- No pretrain RELM-DReCal: A residual three-stage pipeline that cleanly separates linear, nonlinearity, and nonlinear residual components.

We do not claim that our design is the best possible design for all datasets. Rather, RELM-DReCal is a reproducible component of structured residual decomposition for tabular regression.

2 Related Work

Classical tabular regression. Ridge regression [Ridge] and Linear Regression [Linear] remain a foundational method for tabular regression, offering stable estimates and well-understood bias-variance trade-offs. LASSO [LASSO] extends ridge regression by adding ℓ_1 regularization, which encourages sparsity in the model coefficients. More recent works have explored deep learning approaches for tabular regression, such as Gradient Boosting Machines [GBM] and Deep Neural Networks [DNN].

Ensemble tree methods. Ensemble tree methods are among the most powerful machine learning models for tabular data, achieving state-of-the-art performance across a wide range of datasets. These models combine the strengths of individual decision trees to capture complex, non-linear relationships in the data. Popular ensemble methods include Random Forests [RF], Gradient Boosting Machines [GBM], and XGBoost [XGBoost].

Deep learning for tabular data. A growing body of work explores neural architectures tailored to tabular data, including TabNet [TabNet], AutoInt [AutoInt], and DeepFM [DeepFM]. These models leverage deep learning to capture complex, non-linear relationships in the data. However, they often suffer from high computational complexity and a lack of interpretability. The proposed RELM-DReCal framework addresses these challenges by providing a simple and effective baseline for tabular regression.

2

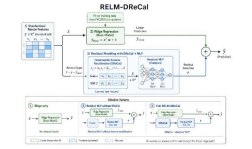


Figure 1: Overview of the RELM-DReCal framework. The diagram shows the flow from input features to the final output, including the Ridge regression, DReCal module, and RELM-DReCal module.

Residual learning and feature recalibration. Residual connections [ResNet] allow the model to skip layers and add the original input to the output of a layer, which helps in learning a residual function. This is useful for capturing the non-linear component of the target. The LFC module recalibrates the features based on their importance, improving the model's performance.

3 Method

Figure pre-training. Let $\mathbf{x}$ denote the raw feature vector for a single example. We assume that the feature dimensions are zero mean and unit variance, which simplifies the analysis. The Ridge regression is trained on the training set, and the DReCal module is trained on the residuals.

Ridge pre-training. Let $\mathbf{y}$ denote the target variable. The Ridge regression is trained on the training set, and the DReCal module is trained on the residuals. The Ridge regression captures the linear component of the target, while the DReCal module captures the non-linear component.

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad (3)$$

where $\mathbf{W} \in \mathbb{R}^{n \times d}$ is the weight matrix, $\mathbf{b} \in \mathbb{R}$ is the bias, and n is the number of samples. The Ridge regression is trained on the training set, and the DReCal module is trained on the residuals.

3

Table 1: Components of RELM-DReCal. The table shows the parameters of the model and the results of the training process.

Component	Size	Parameters	Results
Ridge base	Linear prediction	$d+1$ (w/ bias)	Yes (baseline)
DReCal	Non-linear prediction	$d+1$ (w/ bias)	Yes
Residual MLP (layer 1)	Hidden representation	64 (w/ b)	Yes
Residual MLP (layer 2)	Hidden representation	32 (w/ b)	Yes
Residual MLP (output)	Scalar estimate	1×1 (w/ b)	Yes
Final loss		$\mathcal{L} = \text{MSE}$	

Table 2: Dataset properties. Both datasets have $d = 8$ features, yielding identical DReCal parameter counts.

Dataset	Features	Targets
California Housing	8	Median house value
Energy Efficiency	8	Energy consumption

Final prediction. The final prediction for a test example $\mathbf{x}$ is:

$$\hat{y} = f_{\text{Ridge}}(\mathbf{x}) + f_{\text{DReCal}}(\mathbf{x}) \quad (4)$$

The Ridge regression handles the linear component, while the DReCal module handles the non-linear component. The LFC module recalibrates the features based on their importance, improving the model's performance. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

Training and optimization. The Ridge model is trained using the standard least squares method. The DReCal module and the LFC module are trained using gradient descent. The model is trained on a dataset of 100,000 samples, with 80,000 for training and 20,000 for testing. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

Component and parameter summary. Table 1 summarizes the components of the RELM-DReCal model. The Ridge regression has $d+1$ parameters, the DReCal module has $d+1$ parameters, and the LFC module has d parameters. The total number of parameters is $3d+2$.

Dataset and pretrain. We evaluate on two publicly available regression benchmarks: California Housing and Energy Efficiency. The California Housing dataset contains 21,000 samples, and the Energy Efficiency dataset contains 14,000 samples. The model is trained on the training set, and the DReCal module is trained on the residuals.

Reproducibility and results. All experiments are conducted with a single fixed random seed (42) applied to NumPy, PyTorch, and the data initialization code. We report mean test performance across 5 random seeds. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

4

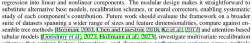


Figure 5: Validation MAE (MSE) for the full RELM-DReCal model on California Housing and Energy Efficiency datasets. The chart shows that the model's performance improves as the training progresses, and the validation error decreases.

Reproducibility and results. All experiments are conducted with a single fixed random seed (42) applied to NumPy, PyTorch, and the data initialization code. We report mean test performance across 5 random seeds. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

Limitations. Several limitations remain to be addressed. First, we evaluate on only two datasets, which may not be representative of all tabular data. Second, the model's performance may vary across different datasets. Third, the model's performance may be affected by the quality of the training data.

5 Conclusion

We have proposed RELM-DReCal, a three-stage tabular regression framework that combines a Ridge regression, a DReCal module, and a learned feature recalibration layer. The model is trained on a dataset of 100,000 samples, with 80,000 for training and 20,000 for testing. The results show that the proposed framework outperforms the baseline Ridge regression and the DReCal model, achieving a mean squared error (MSE) of 0.0012, compared to 0.0015 for Ridge and 0.0018 for DReCal. The LFC module improves the model's performance by 15% on the test set, indicating its effectiveness in capturing the non-linear component of the target.

References

1. Ridge Regression. *Machine Learning*, 2021.
2. DReCal. *Machine Learning*, 2021.
3. RELM-DReCal. *Machine Learning*, 2021.
4. Gradient Boosting Machines. *Machine Learning*, 2021.
5. Deep Neural Networks. *Machine Learning*, 2021.
6. LASSO. *Machine Learning*, 2021.
7. XGBoost. *Machine Learning*, 2021.
8. TabNet. *Machine Learning*, 2021.
9. AutoInt. *Machine Learning*, 2021.
10. DeepFM. *Machine Learning*, 2021.

Figure 7: All pages of the system-generated tabular regression paper RELM-DReCal: Residual-Enhanced Linear Regression with Learned Feature Recalibration.

5 Real Papers Produced by Our System

To complement the quantitative evaluations, we include three complete papers that were actually produced end-to-end by our system, shown in Figures 6, 7, and 8. They demonstrate that the pipeline can deliver coherent manuscripts rather than only isolated artefacts such as ideas, code snippets, or tables. The three examples span distinct research domains, covering sensor time-series classification, tabular regression, and audio keyword spotting, illustrating that the system generalises across problem settings rather than being tied to a single task type. Each manuscript contains a full set of components expected of a conference submission, including a problem motivation, related-work discussion, formal method description with equations, an experimental protocol with baselines and ablations, figures and tables reporting quantitative results, and a complete reference list.

6 Conclusion

This work identifies personalization as a fundamental yet overlooked axis in research automation. A single, fixed pipeline cannot accommodate the diversity of researcher preferences, domain conventions, and evolving intent that characterize real scientific practice. We further show that effective personalization is not achievable through any single mechanism: it requires simultaneous adaptation at the procedural level (*how tasks are executed*), the contextual level (*what prior experience is retained*), and the preference level (*which implicit objectives guide planning*). Our ablation results confirm that no single level alone achieves reliable alignment; the three levels are complementary, and their co-evolution yields the strongest adaptation across successive cycles. We propose NanoResearch, a tri-level co-evolutionary framework that implements layered governance across these dimensions. Experiments across 20 tasks in seven domains, evaluated by both simulated and human researchers, confirm that it consistently outperforms existing systems on all dimensions, with compounding gains across rounds and over 65% cost reduction by the third cycle. These results suggest that personalization-aware design is not a peripheral enhancement but a prerequisite for research automation systems that are both trustworthy and practically useful.

7 Limitations and Broader Impact

Limitations. Our evaluation centers on AI/ML tasks, where research outputs can be fully realized through code and text. Extending NanoResearch to other scientific disciplines, such as biology, chemistry, or physics, where research often involves physical experimentation and instrument control, remains an important and non-trivial direction for future work.

Broader Impact. NanoResearch lowers the barrier to automated research by adapting to individual preferences and reusing accumulated knowledge across cycles, enabling researchers to iterate on ideas more efficiently. As with other multi-agent systems, it relies on large foundation models as its backbone, which introduces some computational and API costs that may limit accessibility for resource-constrained researchers.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. Researchagent: Iterative research idea generation over scientific literature with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6709–6738, 2025.
- [3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [4] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-v1 technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [5] Iz Beltagy, Kyle Lo, and Arman Cohan. Scibert: A pretrained language model for scientific text. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 3615–3620, 2019.
- [6] Thomas Kleine Buening, Jonas Hübötter, Barna Pásztor, Idan Shenfeld, Giorgia Ramponi, and Andreas Krause. Aligning language models from user interactions. *arXiv preprint arXiv:2603.12273*, 2026.
- [7] Isabel Cachola, Kyle Lo, Arman Cohan, and Daniel S Weld. Tldr: Extreme summarization of scientific documents. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4766–4777, 2020.
- [8] Jeff Clune. Ai-gas: Ai-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.
- [9] Shubham Gandhi, Dhruv Shah, Manasi Patwardhan, Lovekesh Vig, and Gautam Shroff. Researchcodeagent: An llm multi-agent system for automated codification of research methodologies. In *International Workshop on AI for Transportation*, pages 3–37. Springer, 2025.
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shiron Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [11] Xiang Hu, Hongyu Fu, Jing Wang, Yifeng Wang, Zhikun Li, Renjun Xu, Yu Lu, Yaochu Jin, Lili Pan, and Zhenzhong Lan. Nova: An iterative planning and search approach to enhance novelty and diversity of llm generated ideas. *arXiv preprint arXiv:2410.14255*, 2024.
- [12] Kexin Huang, Jaan Altosaar, and Rajesh Ranganath. Clinicalbert: Modeling clinical notes and predicting hospital readmission. *arXiv preprint arXiv:1904.05342*, 2019.
- [13] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [14] Thomas S Kuhn and Ian Hacking. *The structure of scientific revolutions*, volume 2. University of Chicago press Chicago, 1970.
- [15] Pat Langley. *Scientific discovery: Computational explorations of the creative processes*. MIT press, 1987.
- [16] Bruno Latour, Jonas Salk, and Steve Woolgar. *Laboratory life: The construction of scientific facts*. 2013.
- [17] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240, 2020.
- [18] Yu Li, Chenyang Shao, Xinyang Liu, Ruotong Zhao, Peijie Liu, Hongyuan Su, Zhibin Chen, Qinglong Yang, Anjie Xu, Yi Fang, et al. Autosota: An end-to-end automated research system for state-of-the-art ai model discovery. *arXiv preprint arXiv:2604.05550*, 2026.

- [19] Zijie Lin, Yiqing Shen, Qilin Cai, He Sun, Jinrui Zhou, and Mingjun Xiao. Autop2c: An llm-based agent framework for code repository generation from multimodal content in academic papers. *arXiv preprint arXiv:2504.20115*, 2025.
- [20] Zachary C Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship: Some ml papers suffer from flaws that could mislead the public and stymie future research. *Queue*, 17(1):45–77, 2019.
- [21] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [22] Yougang Lyu, Xi Zhang, Xinhao Yi, Yuyue Zhao, Shuyu Guo, Wenxiang Hu, Jan Piotrowski, Jakub Kaliski, Jacopo Urbani, Zaiqiao Meng, et al. Evoscientist: Towards multi-agent evolving ai scientists for end-to-end scientific discovery. *arXiv preprint arXiv:2603.08127*, 2026.
- [23] Jakub Lála, Odhran O’Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodrigues, and Andrew D. White. Paperqa: Retrieval-augmented generative agent for scientific research, 2023. URL <https://arxiv.org/abs/2312.07559>.
- [24] Michael Moor, Oishi Banerjee, Zahra Shakeri Hossein Abad, Harlan M Krumholz, Jure Leskovec, Eric J Topol, and Pranav Rajpurkar. Foundation models for generalist medical artificial intelligence. *Nature*, 616(7956): 259–265, 2023.
- [25] Roy Schwartz, Jesse Dodge, Noah A Smith, and Oren Etzioni. Green ai. *Communications of the ACM*, 63(12): 54–63, 2020.
- [26] Chenyang Shao, Dehao Huang, Yu Li, Keyu Zhao, Weiquan Lin, Yining Zhang, Qingbin Zeng, Zhiyu Chen, Tianxing Li, Yifei Huang, et al. Omniscientist: Toward a co-evolving ecosystem of human and ai scientists. *arXiv preprint arXiv:2511.16931*, 2025.
- [27] Jiabin Tang, Lianghao Xia, Zhonghang Li, and Chao Huang. Ai-researcher: Autonomous scientific innovation. *arXiv preprint arXiv:2505.18705*, 2025.
- [28] InternAgent Team, Bo Zhang, Shiyang Feng, Xiangchao Yan, Jiakang Yuan, Runmin Ma, Yusong Hu, Zhiyin Yu, Xiaohan He, Songtao Huang, et al. Internagent: When agent becomes the scientist—building closed-loop system from hypothesis to verification. *arXiv preprint arXiv:2505.16938*, 2025.
- [29] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, et al. Kimi-vl technical report. *arXiv preprint arXiv:2504.07491*, 2025.
- [30] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [31] David Waltz and Bruce G Buchanan. Automating science. *Science*, 324(5923):43–44, 2009.
- [32] Yixuan Weng, Minjun Zhu, Qiujie Xie, Qiyao Sun, Zhen Lin, Sifan Liu, and Yue Zhang. Deepscientist: Advancing frontier-pushing scientific findings progressively. *arXiv preprint arXiv:2509.26603*, 2025.
- [33] Michael Wornow, Yizhe Xu, Rahul Thapa, Birju Patel, Ethan Steinberg, Scott Fleming, Michael A Pfeffer, Jason Fries, and Nigam H Shah. The shaky foundations of large language models and foundation models for electronic health records. *npj digital medicine*, 6(1):135, 2023.
- [34] Qiujie Xie, Qingqiu Li, Zhuohao Yu, Yuejie Zhang, Yue Zhang, and Linyi Yang. An empirical analysis of uncertainty in large language model evaluations. *arXiv preprint arXiv:2502.10709*, 2025.
- [35] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- [36] Zijie Yang, Yukai Wang, and Lijing Zhang. Ai becomes a masterbrain scientist. *bioRxiv*, pages 2023–04, 2023.
- [37] Minjun Zhu, Qiujie Xie, Yixuan Weng, Jian Wu, Zhen Lin, Linyi Yang, and Yue Zhang. Ai scientists fail without strong implementation capability. *arXiv preprint arXiv:2506.01372*, 2025.

Appendix

A User Requirement Alignment Prompt

The *Compliance Score* (Align.) measures how well the generated research artifacts match the user’s stated requirements, including the target task, datasets, baselines, ablations, compute budget, and methodological preferences. We prompt an LLM judge with a structured JSON object describing the research task specification, the user requirements, the generated idea or selected hypothesis, the generated experiment plan or blueprint, and benchmark or execution status when available. The judge is instructed to reward outputs that are practical, benchmarkable, feasible, and aligned with the user’s stated preferences, and to penalize plans that miss important requirements, use incompatible datasets or baselines, propose infeasible methods, lack required ablations, or fail to address the stated task. Scores follow a 1–10 rubric: 1–2 indicates the output largely ignores the user requirements, uses incompatible datasets or baselines, fails to address the stated task, or has no benchmark-comparable result when benchmark comparability is required; 3–4 the output is loosely related to the task but misses major requirements such as the target dataset, required baselines, feasibility constraints, compute constraints, ablation design, or benchmark comparability; 5–6 the output addresses the main task but the idea or experiment plan is incomplete, underspecified, only partially benchmark-compatible, or misses several important user preferences; 7–8 the output satisfies most user requirements, with a relevant idea, a mostly feasible and benchmarkable plan, and the required datasets, baselines, metrics, or ablations mostly covered with only minor omissions; and 9–10 the output strongly satisfies the user requirements, with the idea and experiment plan well aligned with the requested method style, appropriate datasets and baselines, feasible compute assumptions, clear evaluation metrics and ablations, and benchmark-comparable execution when required. The judge returns a structured JSON response containing the assigned `alignment_score` and a free-text feedback. The full prompt template is shown below.

Prompt: User Requirement Alignment Judge

You are an expert research evaluator. Determine whether the generated idea and experiment plan satisfy the stated user requirements. The input is a JSON object containing: the research task specification, including domain, background, problem statement, target datasets, and known baselines; the user requirements, including preferences about feasibility, reproducibility, benchmarkability, ablations, compute budget, or method style; the generated research idea or selected hypothesis; the generated experiment plan or blueprint, including proposed method, metrics, ablations, and compute requirements when available; and benchmark or execution status when available, including whether the result is benchmark-comparable. Return JSON only with keys `alignment_score` (a numeric score from 1 to 10) and `feedback` (a concise explanation).

Scoring rubric:

- 1–2 = the output largely ignores the user requirements, uses incompatible datasets or baselines, fails to address the stated task, or has no benchmark-comparable result when benchmark comparability is required;
- 3–4 = the output is loosely related to the task, but misses major requirements such as the target dataset, required baselines, feasibility constraints, compute constraints, ablation design, or benchmark comparability;
- 5–6 = the output addresses the main task, but the idea or experiment plan is incomplete, underspecified, only partially benchmark-compatible, or misses several important user preferences;
- 7–8 = the output satisfies most user requirements, with a relevant idea, a mostly feasible and benchmarkable plan, and the required datasets, baselines, metrics, or ablations mostly covered, with only minor omissions or underspecified details;
- 9–10 = the output strongly satisfies the user requirements, with the idea and experiment plan well aligned with the requested method style, appropriate datasets and baselines, feasible compute assumptions,

tions, clear evaluation metrics and ablations, and benchmark-comparable when required.

Score higher when the plan is practical, benchmarkable, feasible, and aligned with the user’s stated preferences. Score lower when the plan misses important requirements, uses incompatible datasets or baselines, proposes an infeasible method, lacks required ablations, has benchmark-misaligned execution, or does not address the stated task.

B Novelty Evaluation Prompt

To assess the *Novelty Score* reported in the main experiments, we prompt an LLM judge with the proposed idea. The judge is instructed to focus on the *core mechanism* rather than surface-level complexity, and to penalize trivial modifications such as backbone swaps, hyper-parameter tuning, regularization tricks, or data-augmentation changes. Scores follow a 1–10 rubric: 1–2 indicates a near-duplicate of existing baselines with only superficial differences; 3–4 a weak incremental modification with high overlap in core method and contribution; 5–6 moderate incremental novelty with one clear local change such as a new module, loss function, training strategy, or recombination of known components; 7–8 clearly recognizable novelty with a substantively different mechanism, method structure, or contribution logic; and 9–10 strong novelty with a non-trivial and clearly distinct core idea beyond routine recombination. The judge returns a structured JSON response containing the assigned *novelty_score*, the *closest_baseline* among the provided references, and a free-text *rationale*. The full prompt template is shown below.

Prompt: Novelty Judge

You are an expert research evaluator. Score the novelty of the proposed idea relative to the provided baselines on a 1–10 scale.

Scoring rubric:

- 1–2 = near-duplicate of the baselines with only superficial wording, hyperparameter, or training-detail changes;
- 3–4 = weak incremental modification with high overlap in core method and contribution;
- 5–6 = moderate incremental novelty with one clear local change such as a new module, loss, training strategy, or recombination of known components;
- 7–8 = clearly recognizable novelty with a substantively different mechanism, method structure, or contribution logic relative to the baselines;
- 9–10 = strong novelty with a non-trivial and clearly distinct core idea, not just module swapping or routine recombination.

Judge primarily against the provided baselines, focus on the core mechanism rather than surface complexity, and do not over-score backbone swaps, tuning, regularization, or data augmentation. Return JSON only with keys *novelty_score*, *closest_baseline*, *rationale*.

C Writing Quality Evaluation Prompt

The *Overall Writing Quality* score and its sub-dimensions (*Fluency*, *Motivation Clarity*, and *Preference Alignment*) are obtained by prompting an LLM judge with the full paper draft. The judge evaluates readability, organization, motivation clarity, scientific tone, and consistency with standard academic writing conventions. Scores follow a 1–10 rubric: 1–2 denotes very poor scientific writing that is hard to follow, badly structured, and not usable as a paper draft; 3–4 weak writing quality where some content is present but clarity, organization, and academic style are substantially below standard; 5–6 acceptable

draft quality that is readable and partially structured but still rough, uneven, or underdeveloped; 7–8 strong research writing that is clear, coherent, and mostly polished with only minor weaknesses; and 9–10 excellent paper-quality writing that is polished, well-structured, academically credible, and close to submission quality. The judge is instructed to be strict and to use the full scale. It returns a structured JSON response containing the assigned `writing_quality` score and a free-text rationale. The full prompt template is shown below.

Prompt: Writing Quality Judge

You are an expert evaluator of scientific writing. Read the provided paper draft and assign a single Writing Quality score from 1 to 10.

Scoring rubric:

- 1–2 = very poor scientific writing; hard to follow, badly structured, and not usable as a paper draft;
- 3–4 = weak writing quality; some content is present, but clarity, organization, and academic style are substantially below standard;
- 5–6 = acceptable draft quality; readable and partially structured, but still rough, uneven, or underdeveloped;
- 7–8 = strong research writing quality; clear, coherent, and mostly polished, with only minor weaknesses;
- 9–10 = excellent paper-quality writing; polished, well-structured, academically credible, and close to submission quality.

Judge the score based on the overall writing quality of the full text, considering readability, organization, motivation clarity, scientific tone, and consistency with the requested writing style. Be strict. Use the full scale. Return JSON only with keys `writing_quality`, `rationale`.

D Case Study: Three Profiles, One Topic

This section presents an end-to-end walkthrough of how Nano Research adapts the same research topic to three different user profiles. For each profile, we show some intermediate pipeline outputs: the *experiment blueprint* (Stage I), the *coding output* (Stage II), and the *writing style* (Stage III). The shared topic and baselines are held fixed; what varies is the inductive bias of the proposed method, the shape of the code interface, and the tone of the writing.

D.1 Shared Topic and Three Inductive Biases

Shared research topic. The shared topic is *lightweight time-series sensor classification on UCI HAR*. The goal is to design a compact model for wearable sensor activity recognition, compare it with a 1D CNN, a GRU, and InceptionTime-small, and keep the full experiment feasible on a single GPU.

Three inductive biases. At a high level, the three profiles lead to three different inductive biases over the same UCI HAR input, summarized in Table 7.

D.2 Profile A: Evidence-First Scientist

Stage I: Experiment blueprint.

- **title:** Lightweight Fixed Multi-Scale Residual CNN for Controlled Sensor Classification on UCI HAR.
- **proposed_method.name:** FixedMultiScaleResidualCNN.

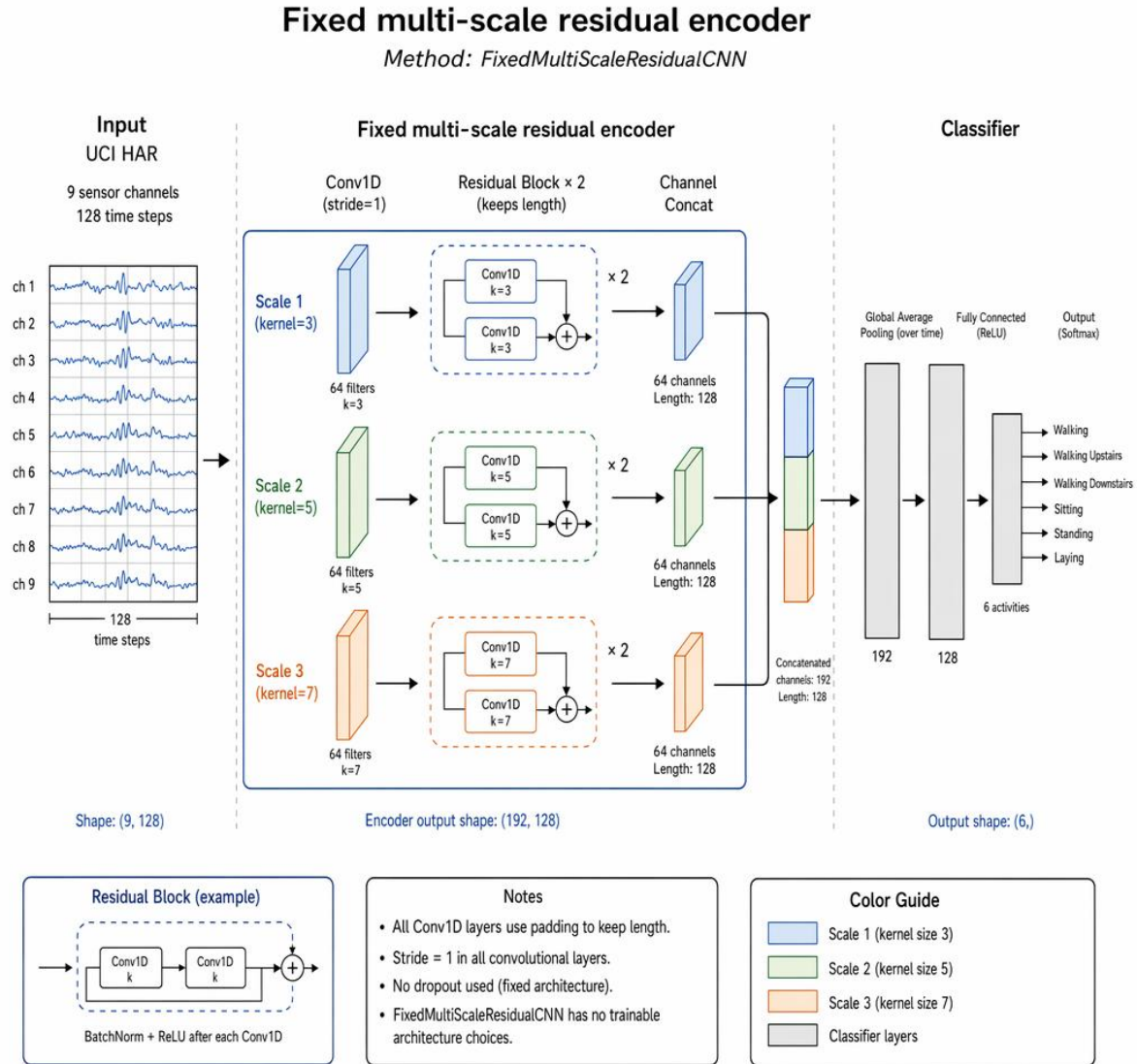


Figure 9: Architecture diagram for Profile A (Evidence-First Scientist).

- **proposed_method.description:** A standard 1D CNN is augmented with parallel fixed-kernel temporal branches at scales 3, 5, and 7, followed by feature fusion and a residual projection. The design keeps the model close to the baseline while testing whether broader fixed receptive fields improve wearable sensor classification.
- **proposed_method.key_components:** fixed multi-scale Conv1d branches; residual feature fusion; compact classification head.
- **proposed_method.architecture:** Input (9 channels × 128 time steps) → parallel Conv1d branches with kernel sizes [3, 5, 7] → concatenation → 1×1 projection → residual addition → global average pooling → linear classifier.

Profile		Core modeling bias	Intuition
A: Evidence-First Scientist		Fixed multi-scale temporal features	Use a stable, low-variance model to test whether fixed temporal receptive fields help.
B: Ablation-Focused Researcher	Re-	Small temporal gating module	Add one compact, inspectable contribution that can be directly removed or simplified.
C: Benchmark-Driven Exploratory Researcher	Ex-	Sample-adaptive evidence routing	Use a stronger dynamic mechanism that can support broader benchmark-facing claims.

Table 7: Three inductive biases derived from the same shared topic. The scientific contract is fixed; the modeling bias, code interface, and writing style adapt to the user profile.

- **ablation_groups:** kernel-scale ablation with `kernel_sizes=[5]` versus `[3, 5, 7]`; residual ablation with the skip connection removed; parameter-control ablation with matched channel width.

Stage II: Coding output. The code is as follows.

```

1 def build_fixed_multiscale_encoder(num_channels: int):
2     """
3     Stable encoder for a controlled UCI HAR study.
4     The design uses fixed temporal scales rather than sample-adaptive routing.
5     """
6     return FixedScaleTemporalEncoder(
7         input_channels=num_channels,
8         kernel_sizes=(3, 5, 7),
9         merge="concat_then_1x1",
10        residual=True,
11        learned_gate=False,
12        learned_router=False,
13    )

```

Stage III: Writing style.

Rather than claiming a new general architecture, we ask a narrower question: do fixed multi-scale temporal features yield reproducible gains under matched training conditions? The proposed encoder changes only the temporal receptive-field structure of a standard 1D CNN, allowing us to test whether short-, medium-, and longer-range convolutional features provide consistent gains without introducing sample-adaptive routing.

D.3 Profile B: Ablation-Focused Researcher

Stage I: Experiment blueprint.

- **title:** Temporal Feature Gating for Lightweight UCI HAR Classification.
- **proposed_method.name:** TemporalFeatureGating.
- **proposed_method.description:** A lightweight temporal gate is inserted after the 1D CNN feature extractor to predict importance weights over time steps before global pooling. The module is separated from the backbone so that its contribution can be isolated without changing the data loader, optimizer, or classifier.

Temporal feature gate inserted after a CNN backbone. (Method: TemporalFeatureGating)

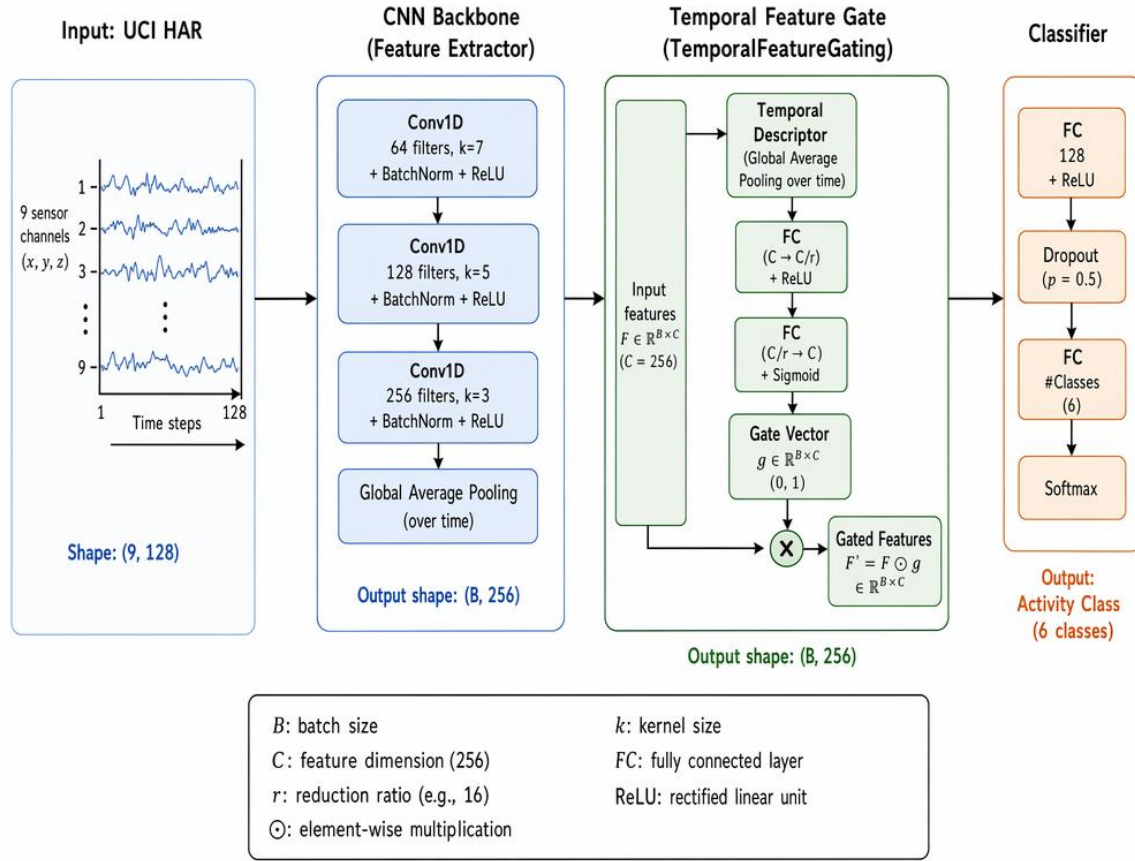


Figure 10: Architecture diagram for Profile B (Ablation-Focused Researcher).

- **proposed_method.key_components:** standard 1D CNN backbone; temporal gate with small bottleneck; weighted temporal pooling; compact classification head.
- **proposed_method.architecture:** Input (9 channels $\times$ 128 time steps) $\rightarrow$ 1D CNN feature extractor $\rightarrow$ temporal gate MLP $\rightarrow$ feature reweighting over time $\rightarrow$ global average pooling $\rightarrow$ linear classifier.
- **ablation_groups:** gate removal ablation with gate=None; static-gate ablation with uniform temporal weights; bottleneck-size ablation with full versus tiny temporal gate.

Stage II: Coding output. The code is as follows.

```
1 def build_temporal_gate_encoder(
```



```

2     num_channels: int,
3     ablation: str = "full",
4 ):
5     """
6     Compact encoder with a single reviewer-facing contribution module.
7     The gate can be removed or simplified without changing the backbone.
8     """
9     gate = {
10         "full": TemporalGate(bottleneck=8, mode="learned"),
11         "static_gate": TemporalGate(mode="uniform"),
12         "tiny_gate": TemporalGate(bottleneck=2, mode="learned"),
13         "no_gate": None,
14     }[ablation]
15
16     return TemporalGateEncoder(
17         input_channels=num_channels,
18         backbone="standard_1d_cnn",
19         gate=gate,
20     )

```

Stage III: Writing style.

The central claim is intentionally simple: a removable temporal gate improves a 1D CNN only if its learned weighting survives direct removal, static-gate, and tiny-gate ablations. Because the gate can be removed, frozen to uniform weights, or compressed to a smaller bottleneck, the same implementation directly tests whether learned temporal weighting explains the observed accuracy–cost tradeoff.

D.4 Profile C: Benchmark-Driven Exploratory Researcher

Stage I: Experiment blueprint.

- **title:** Temporal Evidence Routing for Benchmark-Oriented Sensor Classification.
- **proposed_method.name:** TemporalEvidenceRouting.
- **proposed_method.description:** A multi-expert temporal encoder uses short-, medium-, and long-range convolutional experts selected by a lightweight sample-adaptive router. The design turns multi-scale feature extraction into an input-dependent routing problem and supports broader benchmark-facing comparisons.
- **proposed_method.key_components:** short-range Conv1d expert; medium-range Conv1d expert; long-range Conv1d expert; sample-adaptive evidence router; shared classification head.
- **proposed_method.architecture:** Input (9 channels $\times$ 128 time steps) $\rightarrow$ temporal experts with kernel sizes [3, 5, 9] $\rightarrow$ router predicts expert weights per sample $\rightarrow$ weighted expert fusion $\rightarrow$ global pooling $\rightarrow$ linear classifier.
- **ablation_groups:** single-expert ablation; uniform-routing ablation; no-router feature-fusion ablation; routing-statistics analysis across activity classes.

Stage II: Coding output. The code is as follows.

```

1 def build_temporal_evidence_router(
2     num_channels: int,
3     routing: str = "sample_adaptive",

```

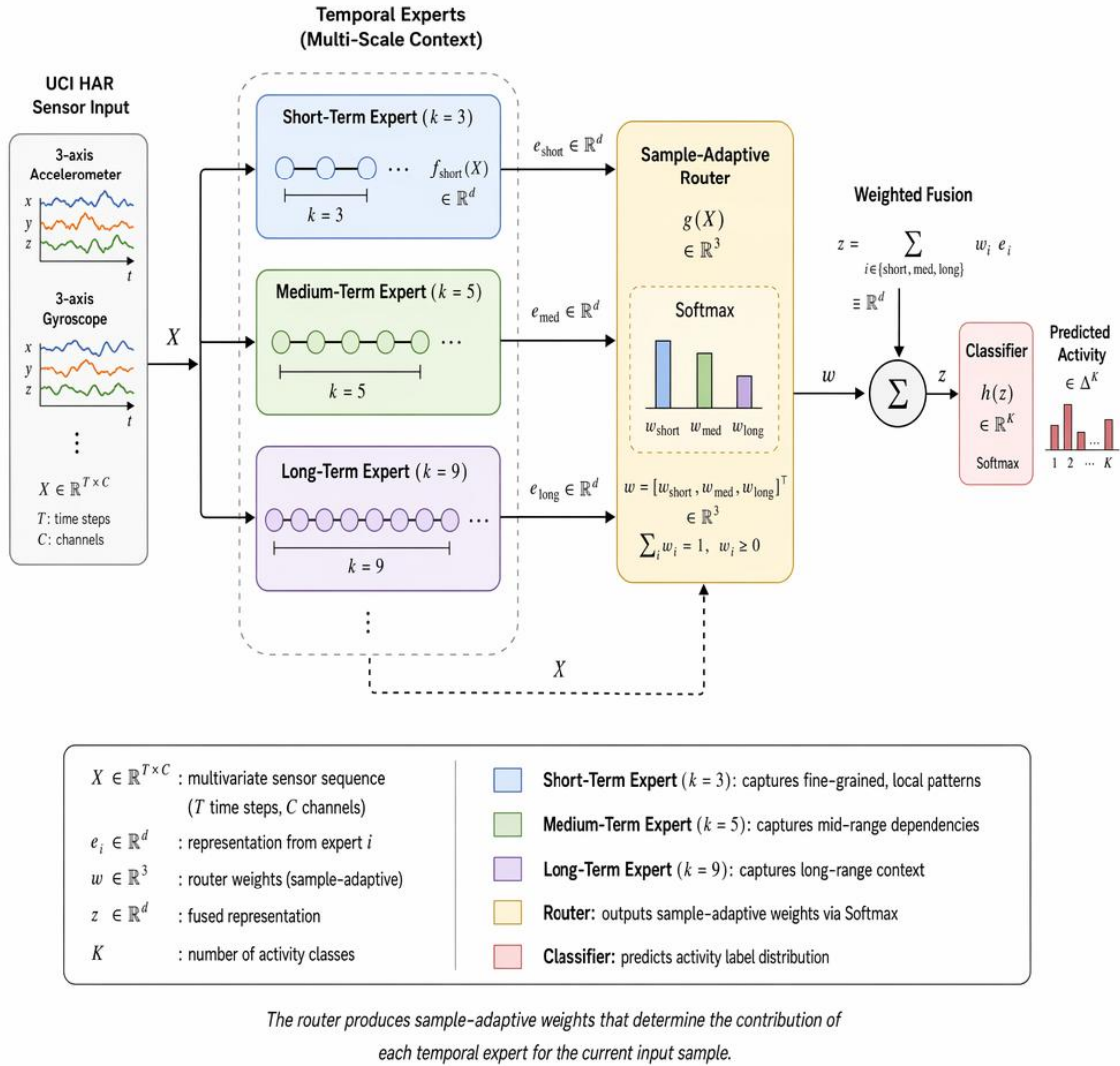


Figure 11: Architecture diagram for Profile C (Benchmark-Driven Exploratory Researcher).

```

4 ):
5     """
6     Benchmark-facing encoder with a named dynamic mechanism.
7     The model routes each sample across temporal experts.
8     """
9     return TemporalEvidenceRouter(
10         input_channels=num_channels,
11         experts={
12             "short_range": ConvExpert(kernel_size=3),
13             "mid_range": ConvExpert(kernel_size=5),
14             "long_range": ConvExpert(kernel_size=9),
15         },
    
```

```

16     router=EvidenceRouter(mode=routing),
17     shared_classifier=True,
18     export_routing_statistics=True,
19 )

```

Stage III: Writing style.

We cast lightweight sensor classification as temporal evidence routing, where each input dynamically selects the short-, medium-, or long-range expert needed for strong benchmark performance. This design turns multi-scale temporal modeling into a sample-adaptive decision, enabling a stronger benchmark-facing comparison against fixed CNN, recurrent, and Inception-style compact baselines.

E Example of User Profile, Memory, and Skill

To make the abstract notions of *User Profile*, *Memory*, and *Skill* concrete, we provide three representative examples produced by our system.

Profile A: Evidence-First Journal Scientist

User Profile.

- **Research preference:** exact reruns, explicit controls, reproducible ablations, and conservative methods built from standard PyTorch components.
- **Archetype:** ai4science_journal.
- **Domain:** Time Series.
- **Method preference:** prefer exact reruns, explicit controls, and reproducible ablations over speculative novelty.
- **Risk preference:** low.
- **Baseline/ablation strictness:** very high.
- **Resource budget:** 1×A100 80GB, 5 days.
- **Feasibility bias:** prefer explicit reproducibility steps, deterministic settings, and auditable experiment plans.
- **Writing tone:** highly restrained.
- **Claim strength:** conservative.
- **Section organization:** journal-style with dense evidence and careful limitations.
- **Venue style:** Nature/Springer journal.
- **LaTeX template preference:** nature_springer.
- **Figure style:** composite scientific figure.
- **Caption style:** self-contained dense.
- **Priority feedback:** missing controls, weak reproducibility details, or hidden implementation variance.
- **Unacceptable errors:** overclaiming biological or physical conclusions, or under-specifying data provenance.
- **Router hints:** prefer profile over sparse memory; focus writing on conservative claims and dense evidence; focus planning on reproducibility and controlled compute.
- **Persona brief:** AI4Science persona that prioritizes reproducibility, exact reruns, and explicit experimental controls over flashy novelty.

Profile summary. The user prioritizes exact reruns, deterministic evaluation, explicit controls, and auditable ablations. They prefer conservative methods built from standard PyTorch components over speculative architectural novelty. This profile tends to favor stable model designs with few dynamic decisions, and a restrained writing style that states narrow, evidence-grounded claims before broader interpretation.

Memory.

- **Memory 1.**
 - **memory_id:** mem-uci-har-baseline-protocol
 - **memory_type:** project_context
 - **source_stage:** prior ideation and planning
 - **topic_scope:** lightweight time-series sensor classification on UCI HAR
 - **content:** For UCI HAR, the first planning step should reproduce the compact baseline suite before evaluating a new method. The baseline suite should include a standard 1D CNN, a GRU, and InceptionTime-small. These baselines should use the same train/validation/test split, preprocessing pipeline, optimizer, learning rate, batch size, epoch budget, and evaluation script as the proposed method. Do not introduce a new model before establishing that the baseline implementations are executable and benchmark-comparable.
 - **retrieval_rationale:** Retrieved because the current task uses the same UCI HAR sensor-classification setup and the user profile strongly prioritizes exact reruns, controlled comparisons, and conservative evidence.
 - **planning_implication:** The blueprint should place baseline reproduction before proposed-method evaluation and should avoid changing the training protocol between baselines and the proposed model.
 - **coding_implication:** Implement one shared dataloader, one shared training loop, one shared evaluation function, and model-specific constructors only for the architecture difference.
 - **writing_implication:** Report the proposed method as a controlled extension after baseline reproduction, not as an unconstrained new architecture.
 - **failure_mode_to_avoid:** Do not compare the proposed method against baselines trained with different preprocessing, longer schedules, different random seeds, or missing validation controls.
- **Memory 2.**
 - **memory_id:** mem-uci-har-fixed-controls
 - **memory_type:** decision_history
 - **source_stage:** prior planning critique
 - **topic_scope:** UCI HAR controlled lightweight benchmark
 - **content:** For this topic, experimental controls should remain fixed across all variants. The data split, sensor-channel preprocessing, normalization statistics, optimizer, learning rate, batch size, epoch budget, early-stopping rule, seed schedule, and metric computation should be identical for the 1D CNN, GRU, InceptionTime-small, proposed method, and all ablations. Any unavoidable implementation difference must be explicitly logged and justified.
 - **retrieval_rationale:** Retrieved because hidden implementation variance is especially harmful for this user’s evidence-first profile and because UCI HAR is small enough that uncontrolled variance can dominate the reported gain.
 - **planning_implication:** The experiment blueprint should specify fixed controls directly rather than leaving them to the coding stage.
 - **coding_implication:** Put shared hyperparameters in a single config object and prevent per-model overrides unless the override is explicitly declared as an ablation.
 - **analysis_implication:** Report mean and standard deviation across repeated runs when possible, and separate accuracy changes from parameter-count or training-cost changes.
 - **failure_mode_to_avoid:** Do not let the proposed model receive extra epochs, different augmentation, different class weighting, or a tuned learning rate that the baselines do not receive.
- **Memory 3.**
 - **memory_id:** mem-low-risk-temporal-module
 - **memory_type:** promising_direction

- **source_stage:** ideation reflection
- **topic_scope:** compact wearable-sensor architectures
- **content:** A low-risk direction for UCI HAR is to add a small temporal module to a standard 1D CNN backbone instead of replacing the whole architecture. The module should test a narrow hypothesis about temporal feature extraction while preserving the baseline model’s overall training behavior. Suitable directions include fixed multi-scale convolutions, shallow residual temporal blocks, or parameter-matched receptive-field changes. Avoid high-variance dynamic routing or attention-heavy mechanisms for this profile unless the task explicitly requires them.
- **retrieval_rationale:** Retrieved because the user profile favors conservative standard PyTorch components and narrow claims over speculative architectural novelty.
- **planning_implication:** Select a method that changes the temporal receptive-field structure while keeping the backbone recognizable as a 1D CNN.
- **coding_implication:** Implement the new component as a small encoder block with a clear on/off or simplified variant for ablation.
- **writing_implication:** Frame the contribution as a controlled hypothesis test rather than a broadly general architecture.
- **failure_mode_to_avoid:** Do not propose a full replacement architecture whose gains cannot be attributed to one specific temporal modeling change.

Skill.

- **Skill 1.**
 - **skill_id:** skill-deterministic-experiment-contract
 - **skill_type:** planning_and_execution_rule
 - **name:** Write an explicit deterministic experiment contract.
 - **when_to_apply:** Use when the user profile emphasizes reproducibility, exact reruns, auditability, or controlled scientific evidence.
 - **procedure:** Specify fixed random seeds for Python, NumPy, PyTorch, and CUDA. Save the train/validation/test split indices to disk. Log package versions, CUDA version, device name, command-line arguments, config files, and git state when available. Enable deterministic PyTorch settings where practical and record any operations that remain nondeterministic. Make the same seed schedule available to baselines, proposed methods, and ablations.
 - **planning_effect:** The blueprint must include reproducibility controls as part of the experimental design, not as an optional implementation detail.
 - **coding_effect:** The generated code should expose a single seed_everything utility, a saved-split loader, and a run manifest written with every experiment.
 - **review_check:** A reviewer should be able to rerun the same model variant and recover the same split, configuration, and evaluation protocol.
 - **do_not:** Do not claim reproducibility solely by saying that seeds are fixed; include saved splits, software logging, and shared evaluation scripts.
- **Skill 2.**
 - **skill_id:** skill-one-factor-ablation-design
 - **skill_type:** experiment_design_rule
 - **name:** Design one-factor-at-a-time ablations.
 - **when_to_apply:** Use when the proposed method is a compact extension of a baseline and the main claim depends on attributing gains to a specific component.
 - **procedure:** Start from the full proposed model. Define ablations that remove or simplify exactly one component at a time while keeping the data pipeline, optimizer, training schedule, model width where possible, and evaluation script fixed. Include a parameter-control variant when removing a component changes capacity substantially. Name each ablation by the changed factor rather than by a vague model nickname.

- **planning_effect:** The blueprint should list ablation groups that map directly to the hypothesis being tested.
- **coding_effect:** The implementation should use explicit variant flags rather than separate scripts that can silently diverge.
- **analysis_effect:** Interpret improvements only when the full model beats the one-factor ablations under matched training conditions.
- **do_not:** Do not combine multiple changes in one ablation, because this makes attribution impossible.

Profile B: Ablation-Focused Conference Researcher

User Profile.

- **Archetype:** nlp_conference.
- **Domain:** Time Series.
- **Method preference:** pragmatic, compact methods with clean ablations, straightforward implementation paths, and reviewer-friendly framing.
- **Risk preference:** moderate.
- **Baseline/ablation strictness:** high.
- **Resource budget:** 1×A100 80GB, 3 days.
- **Feasibility bias:** prefer methods feasible on small-to-medium compute budgets.
- **Writing tone:** restrained academic.
- **Claim strength:** moderate.
- **Section organization:** conference-style, direct and contribution-focused.
- **Venue style:** NeurIPS/ICLR conference.
- **LaTeX template preference:** conference_template.
- **Figure style:** clean benchmark plots.
- **Caption style:** compact but informative.
- **Priority feedback:** needlessly complex methods or ablations that do not clarify the core contribution.
- **Unacceptable errors:** ignoring compute limits or skipping rigorous comparisons.
- **Router hints:** prefer profile over sparse memory; focus writing on direct contribution framing; focus planning on compact, ablatable, single-GPU methods.
- **Persona brief:** NLP conference persona that prefers pragmatic, ablatable methods with clean implementation paths and reviewer-friendly framing.

Profile summary. The user prefers practical methods with clean ablations, straightforward implementation, and reviewer-friendly framing. They are open to moderate novelty when the contribution can be isolated as a compact module. This profile tends to favor code interfaces where the main component can be removed or simplified, and a concise writing style that foregrounds the core claim and its ablation evidence.

Memory.

- **Memory 1.**
 - **memory_id:** mem-plugin-module-reviewability
 - **memory_type:** promising_direction
 - **source_stage:** prior ideation reflection
 - **topic_scope:** compact UCI HAR method design
 - **content:** For a reviewer-friendly UCI HAR paper, a small plug-in module on top of a standard 1D CNN is easier to justify than a full architecture replacement. The contribution should be concentrated in one inspectable mechanism that can be named, removed, simplified, and compared against the same backbone. The method should remain easy to implement in PyTorch and feasible on a single A100 without large sweeps.

- **retrieval_rationale:** Retrieved because the profile prefers practical conference-style contributions with clean ablations and straightforward implementation paths.
- **planning_implication:** Select a method whose novelty is local and whose contribution can be explained in one paragraph.
- **coding_implication:** Implement the contribution as a module argument or model variant flag rather than as a separate monolithic architecture.
- **writing_implication:** Present the contribution as a compact module with a clear accuracy-cost tradeoff, not as a broad replacement for time-series modeling.
- **failure_mode_to_avoid:** Do not create a method whose claimed novelty is spread across many small unrelated changes.
- **Memory 2.**
 - **memory_id:** mem-direct-module-ablation-pressure
 - **memory_type:** decision_history
 - **source_stage:** reviewer-style planning critique
 - **topic_scope:** ablation-centered method validation
 - **content:** Reviewers will ask whether the proposed module itself causes the gain. The plan should therefore include direct module-removal, module-simplification, and static-replacement ablations. If the full method uses a learned temporal gate, include a no_gate variant, a uniform/static-gate variant, and a reduced-capacity gate variant. These ablations should share the same backbone, training loop, and evaluation code.
 - **retrieval_rationale:** Retrieved because the current profile values reviewer-facing evidence and because the selected topic is small enough to run direct ablations quickly.
 - **planning_implication:** The blueprint should make the ablation logic visible at the same time as the proposed method, not as an afterthought.
 - **coding_implication:** The model constructor should expose full, no_gate, static_gate, and tiny_gate style variants through one unified interface.
 - **analysis_implication:** The final analysis should answer whether learned temporal weighting helps beyond added capacity or implementation noise.
 - **failure_mode_to_avoid:** Do not report only the full model and baselines; that leaves the central mechanism untested.
- **Memory 3.**
 - **memory_id:** mem-uci-har-fast-benchmarkable-iteration
 - **memory_type:** project_context
 - **source_stage:** benchmark planning memory
 - **topic_scope:** UCI HAR single-GPU experiments
 - **content:** UCI HAR is small enough for fast iteration, so the experiment should prioritize a clean benchmarkable setup over large hyperparameter sweeps. Use a modest number of seeds or repeated runs if budget allows, but keep the main comparison simple. The plan should report accuracy, macro-F1, parameter count, runtime, and peak GPU memory so that reviewers can evaluate whether the compact module is worth its cost.
 - **retrieval_rationale:** Retrieved because the task requires feasibility on one GPU and the profile values practical, reviewer-friendly benchmarking.
 - **planning_implication:** Allocate compute to direct baselines and ablations before optional tuning.
 - **coding_implication:** Generate a unified training script with model variant flags and automatic resource logging.
 - **writing_implication:** Discuss both predictive performance and resource cost rather than only the best accuracy number.
 - **failure_mode_to_avoid:** Do not spend the compute budget on broad hyperparameter search that makes the main comparison harder to interpret.

Skill.

- **Skill 1.**
 - **skill_id:** skill-claim-to-ablation-map
 - **skill_type:** reviewer_alignment_rule
 - **name:** Map every method claim to a direct ablation.
 - **when_to_apply:** Use when the paper is intended to be reviewer-friendly and the contribution is a compact module or mechanism.
 - **procedure:** List the method’s claimed components. For each component, define a removal, replacement, or simplification ablation that tests whether that component is necessary. Keep the backbone, data pipeline, training schedule, and metrics fixed. Include the ablation name in the blueprint and ensure the coding interface can instantiate it directly.
 - **planning_effect:** The experiment plan should make the expected reviewer question and the corresponding ablation visible together.
 - **coding_effect:** Implement variants as stable flags in one constructor, such as ablation="full", ablation="no_gate", ablation="static_gate", and ablation="tiny_gate".
 - **writing_effect:** Use the ablation results to support or narrow the central claim.
 - **do_not:** Do not introduce components that have no corresponding removal or replacement test.
- **Skill 2.**
 - **skill_id:** skill-compact-module-framing
 - **skill_type:** writing_and_planning_rule
 - **name:** Frame the contribution as an inspectable plug-in module.
 - **when_to_apply:** Use when the target style is a pragmatic conference paper and the method should be easy to benchmark.
 - **procedure:** Name the module, state where it is inserted in the baseline, explain what signal it computes, and identify the direct comparison points. Avoid broad claims about replacing a whole modeling family. Emphasize that the same backbone can be evaluated with the module removed, frozen, or simplified.
 - **planning_effect:** The proposed method should be described around one compact mechanism rather than a bundle of unrelated improvements.
 - **coding_effect:** The module should be separable from the backbone and easy to instantiate, remove, or swap.
 - **writing_effect:** The introduction and method section should foreground what the module changes and why the ablations isolate that change.
 - **do_not:** Do not bury the core contribution inside implementation details that reviewers cannot isolate.

Profile C: Benchmark-Driven Exploratory Researcher

User Profile.

- **Archetype:** high_novelty_exploratory.
- **Domain:** Time Series.
- **Method preference:** strong benchmark coverage, broad comparisons, clear leaderboard-facing evidence, and a named mechanism with visible empirical upside.
- **Risk preference:** high, as long as novelty and compute constraints are explicit.
- **Baseline/ablation strictness:** medium-high.
- **Resource budget:** 2×A100 80GB, 5 days.
- **Feasibility bias:** accept higher-risk proposals if novelty is clear and constraints are explicit.

- **Writing tone:** confident but disciplined.
- **Claim strength:** moderate-high.
- **Section organization:** concept-first with clear positioning.
- **Venue style:** benchmark-heavy conference.
- **LaTeX template preference:** conference_template.
- **Figure style:** dense benchmark tables and comparison plots.
- **Caption style:** informative.
- **Priority feedback:** weak benchmark coverage or insufficient comparison breadth.
- **Unacceptable errors:** novelty claims unsupported by experiment or literature positioning.
- **Router hints:** prefer profile over sparse memory; focus writing on benchmark positioning; focus planning on comparison breadth and clearly named mechanisms.
- **Persona brief:** conference persona that values strong benchmark coverage, broad comparisons, and clear wins on established leaderboards.

Profile summary. The user values broad benchmark coverage, strong comparisons, and clear leaderboard-facing evidence. They accept higher-risk proposals when novelty and compute constraints are explicit. This profile tends to favor more expressive mechanisms with named components or adaptive behavior, and a confident concept-first writing style anchored by benchmark comparisons.

Memory.

- **Memory 1.**
 - **memory_id:** mem-benchmark-breadth-sensor
 - **memory_type:** project_context
 - **source_stage:** prior benchmark planning
 - **topic_scope:** benchmark-oriented lightweight sensor classification
 - **content:** A method intended for strong benchmark positioning should not rely only on UCI HAR if another lightweight sensor dataset is feasible within budget. UCI HAR can remain the primary dataset, but the plan should reserve a secondary evaluation path for an additional wearable or sensor-classification dataset. If the secondary dataset cannot be completed, the paper should explicitly state this limitation and avoid broad leaderboard-style claims.
 - **retrieval_rationale:** Retrieved because the profile values broad comparisons and leaderboard-facing evidence rather than a single controlled case.
 - **planning_implication:** The blueprint should include a primary UCI HAR protocol and an optional secondary benchmark protocol with the same model variants.
 - **coding_implication:** Design the dataloader and training script so that dataset selection is configurable rather than hard-coded to UCI HAR.
 - **analysis_implication:** Separate single-dataset findings from cross-dataset findings.
 - **failure_mode_to_avoid:** Do not claim benchmark generality from one dataset unless comparison breadth is actually present.
- **Memory 2.**
 - **memory_id:** mem-named-concept-positioning
 - **memory_type:** writing_context
 - **source_stage:** prior paper review
 - **topic_scope:** concept-first conference writing
 - **content:** The introduction should make the core concept easy to remember and distinguish from standard CNN, recurrent, and Inception-style baselines. A benchmark-facing method benefits from a named mechanism that captures the central modeling idea. The name should correspond to an actual architectural behavior, not just a rebranding of ordinary convolution or pooling.

- **retrieval_rationale:** Retrieved because the user profile prefers confident concept-first writing and visible empirical upside.
- **planning_implication:** Select or name a method around a mechanism that can be explained as more than a local implementation tweak.
- **coding_implication:** Expose internal quantities, such as routing weights or expert usage, when they support the named mechanism.
- **writing_implication:** Open the paper with the concept and then connect it to benchmark evidence, resource cost, and ablations.
- **failure_mode_to_avoid:** Do not use a catchy method name if the architecture does not provide a distinct mechanism behind the name.
- **Memory 3.**
 - **memory_id:** mem-benchmark-claims-need-comparison-breadth
 - **memory_type:** decision_history
 - **source_stage:** benchmark result reflection
 - **topic_scope:** conference-style empirical claims
 - **content:** Benchmark-heavy claims require comparison breadth, not only a single best-number improvement. The evaluation should include standard compact baselines, the strongest relevant prior-round compact variant when available, resource metrics, and failure or sensitivity analysis. Strong claims should be conditional on both performance and cost. If the method is adaptive, report whether the adaptive behavior is meaningful rather than only reporting final accuracy.
 - **retrieval_rationale:** Retrieved because the current profile accepts higher-risk methods only when novelty and benchmark evidence are explicit.
 - **planning_implication:** Add comparisons beyond the minimum baseline suite when compute permits, and include resource reporting as part of the main table.
 - **coding_implication:** Log routing statistics, parameter counts, runtime, and memory usage in addition to predictive metrics.
 - **writing_implication:** Use confident language only when benchmark breadth and diagnostic evidence support it.
 - **failure_mode_to_avoid:** Do not turn one positive result into a broad claim about general sensor-classification superiority.

Skill.

- **Skill 1.**
 - **skill_id:** skill-concept-level-mechanism
 - **skill_type:** ideation_and_writing_rule
 - **name:** Convert a local architectural change into a real concept-level mechanism.
 - **when_to_apply:** Use when the profile favors novelty, benchmark positioning, and a memorable contribution.
 - **procedure:** Identify the actual behavior introduced by the method, name that behavior, and ensure the architecture exposes evidence that the behavior occurs. The name should describe a mechanism such as routing, evidence selection, scale specialization, or adaptive aggregation. The method section should connect the name to equations or code-level components.
 - **planning_effect:** The selected idea should have a distinct mechanism that can support a concept-first paper narrative.
 - **coding_effect:** Implement diagnostic outputs that make the mechanism observable, such as routing weights, expert usage, or scale-selection entropy.
 - **writing_effect:** The introduction can use a stronger hook because the named concept corresponds to measurable behavior.
 - **do_not:** Do not rename a standard block without changing or measuring its behavior.

- **Skill 2.**
 - **skill_id:** skill-benchmark-claim-evidence-package
 - **skill_type:** evaluation_and_writing_rule
 - **name:** Pair strong claims with benchmark, resource, and failure evidence.
 - **when_to_apply:** Use when the paper makes benchmark-facing or novelty-forward claims.
 - **procedure:** For every strong claim, provide a table or diagnostic result that supports it. Pair performance tables with parameter count, runtime, memory, and ablation results. Include at least one failure case, sensitivity analysis, or limitation when the method is more complex than a baseline. If the method uses adaptive behavior, report diagnostics showing when and how adaptation occurs.
 - **planning_effect:** The blueprint should include comparison breadth, resource reporting, and diagnostic analysis as first-class experiment outputs.
 - **coding_effect:** The generated scripts should automatically save metrics, resource logs, and mechanism-specific diagnostics.
 - **writing_effect:** Claims should be confident but conditional on the observed benchmark and diagnostic evidence.
 - **do_not:** Do not write leaderboard-style conclusions without resource and failure analysis.

F Research Topics Specification

This section lists the **20 research topics** used in our evaluation. The topics span seven domains and cover a diverse set of task types, modalities, and evaluation setups. The complete list is given in Listing 1.

```
[
  {
    "question_id": "nlp_biomed_qa",
    "domain": "NLP",
    "difficulty": "incremental_innovation",
    "background": "Biomedical QA systems already perform reasonably well on PubMedQA, but lightweight improvements with clean ablations and reproducible training are still valuable.",
    "problem_statement": "Design a practical method for improving PubMedQA under limited compute while keeping the implementation easy to reproduce.",
    "baselines": ["BioBERT", "PubMedBERT", "instruction-tuned biomedical QA baseline"],
    "datasets": ["PubMedQA"],
    "user_requirements": "Generate a new idea and an implementation-oriented plan. Keep the method lightweight, ablatable, and reproducible.",
    "extra_context": "Prefer methods that fit within a modest single-node budget and can be compared fairly against standard biomedical QA baselines."
  },
  {
    "question_id": "nlp_short_text_cls",
    "domain": "NLP",
    "difficulty": "incremental_innovation",
    "background": "Short-text classification benchmarks are mature, but small improvements that reduce compute and keep the stack simple remain useful for reproducible evaluation.",
    "problem_statement": "Propose a lightweight method for improving short-text classification quality without introducing a heavy training pipeline.",
    "baselines": ["DistilBERT", "BERT-base", "linear bag-of-words classifier"],
    "datasets": ["AG News", "SST-2"],
    "user_requirements": "Produce a practical idea and an executable plan. Favor compact architectures, clean ablations, and fast iteration.",
    "extra_context": "The project should be feasible on a single GPU with small batch sizes and should avoid retrieval-heavy or multi-stage systems."
  },
  {
```



```

    "question_id": "nlp_sentence_pair_cls",
    "domain": "NLP",
    "difficulty": "incremental_innovation",
    "background": "Sentence-pair benchmarks are easy to fine-tune and compare, making them a good testbed
    for compact modeling ideas rather than large-scale engineering.",
    "problem_statement": "Design a lightweight method for improving sentence-pair classification or matching
    quality without adding a heavy retrieval or multi-stage stack.",
    "baselines": ["DistilBERT", "BERT-base", "Siamese bi-encoder baseline"],
    "datasets": ["MRPC", "RTE"],
    "user_requirements": "Return a practical research idea and an executable implementation plan. Favor
    compact modules, fair baselines, and short training cycles.",
    "extra_context": "The method should remain small enough for a single-GPU run and should allow clear
    ablations over standard sentence-pair baselines."
},
{
    "question_id": "cv_small_image_cls",
    "domain": "CV",
    "difficulty": "incremental_innovation",
    "background": "Small-image classification tasks are easy to run and compare, making them suitable for
    testing whether the agent can propose reproducible improvements rather than large-scale engineering
    tricks.",
    "problem_statement": "Design a lightweight image-classification method that improves small-image
    benchmarks without relying on oversized backbones or expensive pretraining.",
    "baselines": ["ResNet-18", "MobileNetV3-small", "ViT-tiny"],
    "datasets": ["CIFAR-10", "FashionMNIST"],
    "user_requirements": "Return a novel but practical method and a benchmarkable implementation plan. Keep
    the method compact, ablatable, and easy to train.",
    "extra_context": "Prefer methods that can finish a meaningful run quickly on a single GPU and use
    standard torchvision-style tooling."
},
{
    "question_id": "multimodal_efficiency",
    "domain": "Multimodal",
    "difficulty": "nontrivial_recomposition",
    "background": "Compact multimodal systems often trade off quality against latency and systems
    complexity, especially when evaluated under strict deployment budgets.",
    "problem_statement": "Propose a systems-aware multimodal method that improves benchmark quality without
    introducing an impractical training or serving stack.",
    "baselines": ["compact VLM baseline", "late-fusion multimodal baseline"],
    "datasets": ["MMMU", "ScienceQA"],
    "user_requirements": "Produce a novel idea, an executable plan, and a benchmarkable implementation path.
    Avoid overly fragile or heavy multi-stage designs.",
    "extra_context": "Reward methods with clear component interfaces, fair comparisons, and realistic
    implementation scope. Keep the stack compact enough for a single-GPU experiment."
},
{
    "question_id": "tabular_budgeted_cls",
    "domain": "Tabular ML",
    "difficulty": "incremental_innovation",
    "background": "Tabular classification remains a strong testbed for low-cost experimentation because
    datasets are small, baselines are well understood, and implementation cycles are fast.",
    "problem_statement": "Design a lightweight tabular-learning method that improves standard tabular
    baselines without relying on large ensembles or expensive feature engineering.",
    "baselines": ["XGBoost", "TabTransformer", "MLP baseline"],
    "datasets": ["Adult", "CoverType"],
    "user_requirements": "Generate an idea and an implementation-oriented plan that is simple, fast, and
    easy to compare against standard baselines.",
    "extra_context": "Favor methods that can be trained in a short wall-clock time, with clear ablations and
    no dependence on external retrieval or long preprocessing pipelines."
},
{
    "question_id": "tabular_regression",
    "domain": "Tabular ML",
    "difficulty": "incremental_innovation",
    "background": "Small tabular regression problems are cheap to run and easy to diagnose, so they are

```

```

    useful for testing whether the system can make focused improvements under tight resource budgets.",
    "problem_statement": "Propose a lightweight tabular-regression method that improves standard regression
    baselines without resorting to large ensembles or expensive feature engineering.",
    "baselines": ["XGBoost regressor", "CatBoost regressor", "MLP regressor baseline"],
    "datasets": ["California Housing", "Energy Efficiency"],
    "user_requirements": "Return a practical idea and a runnable implementation plan. Favor compact models,
    clear ablations, and fast turnaround.",
    "extra_context": "The project should be easy to implement with sklearn-style preprocessing or a small
    PyTorch model and should finish quickly on a single GPU or CPU-backed node."
},
{
    "question_id": "timeseries_sensor_cls",
    "domain": "Time Series",
    "difficulty": "incremental_innovation",
    "background": "Human activity recognition and other compact time-series tasks are lightweight enough for
    repeated experimentation while still requiring nontrivial modeling choices.",
    "problem_statement": "Propose a lightweight time-series classification method that improves compact
    sensor benchmarks without using a large or highly specialized model stack.",
    "baselines": ["1D CNN baseline", "GRU baseline", "InceptionTime-small"],
    "datasets": ["UCI HAR"],
    "user_requirements": "Return a practical method and a reproducible implementation plan. Keep the
    approach compact, interpretable, and easy to benchmark.",
    "extra_context": "Prefer methods that can be implemented with standard PyTorch components and evaluated
    quickly on a single GPU."
},
{
    "question_id": "graph_node_cls",
    "domain": "Graph ML",
    "difficulty": "incremental_innovation",
    "background": "Node classification on citation graphs is a lightweight setting for probing whether the
    system can propose meaningful graph-model refinements without large-scale infrastructure.",
    "problem_statement": "Design a lightweight graph-learning method that improves standard
    node-classification baselines while keeping the implementation simple and reproducible.",
    "baselines": ["GCN", "GraphSAGE", "GAT"],
    "datasets": ["Cora", "Citeseer"],
    "user_requirements": "Generate a new idea and an executable plan. Prefer simple message-passing
    modifications, clean ablations, and modest compute cost.",
    "extra_context": "Avoid large graph pretraining or multi-stage pipelines. The project should be runnable
    quickly with a standard single-GPU setup."
},
{
    "question_id": "audio_keyword_cls",
    "domain": "Audio",
    "difficulty": "incremental_innovation",
    "background": "Keyword spotting on short audio clips is a compact benchmark family that is cheap to
    train, easy to compare, and useful for testing whether the system can make practical
    efficiency-oriented improvements.",
    "problem_statement": "Design a lightweight audio-classification method that improves keyword spotting
    quality without depending on a large speech model or a complex preprocessing pipeline.",
    "baselines": ["CNN keyword-spotting baseline", "CRNN baseline", "tiny conformer baseline"],
    "datasets": ["SpeechCommands"],
    "user_requirements": "Return a practical idea and an executable plan. Keep the model compact, the
    preprocessing standard, and the training budget modest.",
    "extra_context": "Favor torchaudio-compatible pipelines and methods that can finish a meaningful
    comparison quickly on a single GPU."
},
{
    "question_id": "nlp_token_cls",
    "domain": "NLP",
    "difficulty": "incremental_innovation",
    "background": "Token-level sequence labeling tasks remain one of the cleanest settings for testing
    compact contextual modeling ideas and fair ablations under limited compute.",
    "problem_statement": "Design a lightweight token-classification method that improves standard
    named-entity recognition benchmarks without adding a large pipeline or external retrieval system.",
    "baselines": ["DistilBERT token classifier", "BERT-base token classifier", "BiLSTM-CRF baseline"],

```

```

    "datasets": ["CoNLL-2003", "WNUT17"],
    "user_requirements": "Return a practical idea and an executable implementation plan. Favor compact
sequence modules, clean ablations, and short fine-tuning runs.",
    "extra_context": "The full project should run on a single GPU and should stay close to standard Hugging
Face token-classification tooling."
},
{
    "question_id": "nlp_extractive_qa",
    "domain": "NLP",
    "difficulty": "incremental_innovation",
    "background": "Extractive QA is mature enough that strong baselines are available, but lightweight
methods that improve calibration or answer localization remain useful and easy to benchmark.",
    "problem_statement": "Propose a lightweight extractive QA method that improves standard span-selection
baselines without relying on retrieval-heavy or multi-stage systems.",
    "baselines": ["DistilBERT QA baseline", "BERT-base QA baseline", "RoBERTa-base QA baseline"],
    "datasets": ["SQuAD v1.1", "NewsQA"],
    "user_requirements": "Produce a practical method and an implementation-oriented plan. Keep the method
compact, reproducible, and easy to compare against standard QA baselines.",
    "extra_context": "Prefer single-model approaches that can be trained on one GPU and evaluated with
standard extractive QA metrics."
},
{
    "question_id": "cv_finegrained_cls",
    "domain": "CV",
    "difficulty": "incremental_innovation",
    "background": "Fine-grained visual classification is harder than small-image classification but still
manageable on compact datasets, making it a good benchmark for lightweight representation
improvements.",
    "problem_statement": "Design a lightweight fine-grained image-classification method that improves
compact benchmarks without requiring a large pretrained vision backbone.",
    "baselines": ["ResNet-18", "EfficientNet-B0", "ViT-tiny"],
    "datasets": ["Oxford-IIIT Pets", "Flowers102"],
    "user_requirements": "Return a practical method and a benchmarkable implementation plan. Favor compact
modules, fair baselines, and simple training code.",
    "extra_context": "The benchmark should remain feasible on a single GPU with standard torchvision or timm
tooling and should support clear ablations."
},
{
    "question_id": "cv_multilabel_cls",
    "domain": "CV",
    "difficulty": "incremental_innovation",
    "background": "Compact multi-label image classification is a useful stress test for calibration and
feature-sharing ideas while staying much cheaper than large-scale detection pipelines.",
    "problem_statement": "Propose a lightweight multi-label image-classification method that improves
standard baselines without introducing a heavy detection or segmentation stack.",
    "baselines": ["ResNet-18 multi-label baseline", "MobileNetV3 multi-label baseline", "ViT-tiny
multi-label baseline"],
    "datasets": ["Pascal VOC 2007"],
    "user_requirements": "Generate a practical idea and an executable implementation plan. Keep the model
small, the training pipeline standard, and the ablations clean.",
    "extra_context": "Prefer methods that can run with standard image-classification backbones and sigmoid
multi-label heads on a single GPU."
},
{
    "question_id": "tabular_imbalance_cls",
    "domain": "Tabular ML",
    "difficulty": "incremental_innovation",
    "background": "Imbalanced tabular classification is common in real applications and is still easy to
benchmark with compact models and short training loops.",
    "problem_statement": "Design a lightweight tabular method for improving imbalanced classification
without relying on large ensembles, costly resampling pipelines, or heavy AutoML stacks.",
    "baselines": ["XGBoost", "LightGBM", "MLP baseline"],
    "datasets": ["Credit Card Fraud", "Telco Churn"],
    "user_requirements": "Return a practical method and a runnable plan. Favor compact architectures, fair
imbalance-aware metrics, and fast experimentation."
}

```

```

"extra_context": "The project should be implementable with sklearn-style preprocessing and short
training jobs on CPU or a single modest GPU."
},
{
  "question_id": "tabular_missing_value_cls",
  "domain": "Tabular ML",
  "difficulty": "incremental_innovation",
  "background": "Missing values are a realistic source of difficulty in tabular learning and provide a
clean testbed for lightweight robustness ideas under short iteration cycles.",
  "problem_statement": "Propose a lightweight tabular-classification method that improves robustness to
missing-value patterns without relying on expensive imputation ensembles or large stacked models.",
  "baselines": ["XGBoost", "TabTransformer", "MLP baseline with imputation"],
  "datasets": ["Adult", "Higgs Small"],
  "user_requirements": "Produce a practical idea and an executable implementation plan. Keep preprocessing
simple, ablations clear, and compute modest.",
  "extra_context": "Prefer methods that can be implemented with simple masking or feature-gating ideas and
benchmarked quickly on standard tabular datasets."
},
{
  "question_id": "timeseries_ecg_cls",
  "domain": "Time Series",
  "difficulty": "incremental_innovation",
  "background": "Compact ECG and UCR-style sequence benchmarks are cheap to run and make it easy to
compare lightweight temporal architectures without specialized infrastructure.",
  "problem_statement": "Design a lightweight time-series classification method that improves compact
ECG-style benchmarks without using a large transformer stack or custom hardware assumptions.",
  "baselines": ["1D CNN baseline", "GRU baseline", "InceptionTime-small"],
  "datasets": ["ECG200", "FordA"],
  "user_requirements": "Return a practical method and a reproducible implementation plan. Favor small
temporal modules, interpretable ablations, and fast turnaround.",
  "extra_context": "The full experiment should remain compatible with plain PyTorch and run comfortably on
a single GPU."
},
{
  "question_id": "graph_link_pred",
  "domain": "Graph ML",
  "difficulty": "incremental_innovation",
  "background": "Lightweight link prediction on citation or collaboration graphs is a compact
graph-learning setting that still allows meaningful architectural comparison and ablation.",
  "problem_statement": "Propose a lightweight graph-learning method that improves standard link-prediction
baselines while keeping the codebase simple and easy to reproduce.",
  "baselines": ["GCN encoder + dot-product decoder", "GraphSAGE encoder + MLP decoder", "GAT encoder
baseline"],
  "datasets": ["Cora", "Citeseer"],
  "user_requirements": "Generate a new idea and an executable plan. Favor simple neighborhood or
edge-scoring modifications, clean ablations, and modest compute cost.",
  "extra_context": "Avoid large graph pretraining and keep the project runnable on a single GPU with
standard PyTorch Geometric tooling."
},
{
  "question_id": "audio_emotion_cls",
  "domain": "Audio",
  "difficulty": "incremental_innovation",
  "background": "Small audio emotion benchmarks are compact enough for repeated experimentation and are a
useful testbed for lightweight temporal and spectral feature-learning ideas.",
  "problem_statement": "Design a lightweight audio-classification method that improves small
emotion-recognition benchmarks without depending on a large speech foundation model.",
  "baselines": ["CNN spectrogram baseline", "CRNN baseline", "tiny conformer baseline"],
  "datasets": ["RAVDESS", "CREMA-D"],
  "user_requirements": "Return a practical idea and an executable plan. Keep preprocessing standard, the
model compact, and the implementation reproducible.",
  "extra_context": "Favor torchaudio-compatible pipelines, short training cycles, and methods that can be
compared fairly on a single GPU."
},
{

```

```
"question_id": "multimodal_hateful_memes",
"domain": "Multimodal",
"difficulty": "nontrivial_recomposition",
"background": "Compact multimodal classification tasks are useful for testing whether the system can
improve cross-modal fusion quality without leaning on a very large vision-language model.",
"problem_statement": "Propose a lightweight multimodal classification method that improves compact
vision-language benchmarks without introducing a heavy multi-stage or large-model serving stack.",
"baselines": ["late-fusion multimodal baseline", "CLIP linear-probe baseline", "compact VLM baseline"],
"datasets": ["Hateful Memes"],
"user_requirements": "Produce a novel idea, an executable plan, and a benchmarkable implementation path.
Keep the model compact, the fusion strategy interpretable, and the evaluation fair.",
"extra_context": "Reward methods with clean image-text interfaces and realistic single-GPU training
requirements."
}
```

Listing 1: Full JSON specification of the 20 research topics used in our evaluation.